

操作系统

何柯

2026 年 1 月 5 日

目录

1	操作系统基础知识	2
1.1	基本概念	2
1.2	操作系统的历史演化	2
1.3	操作系统分类	3
1.4	操作系统结构	4
1.5	操作系统的启动和导引	6
1.6	应用程序的执行流程	7
2	操作系统运行环境与运行机制	8
2.1	硬件支持	8
2.2	中断与异常	9
2.3	系统调用	10
3	进程/线程模型	11
3.1	进程 (Process)	11
3.1.1	相关概念	11
3.1.2	状态变迁	12
3.1.3	进程控制	14
3.2	线程	16
3.2.1	相关概念	16
3.2.2	线程与进程的关系	17

3.2.3	线程的实现方式:	17
4	进程/线程调度	18
4.1	基本概念	18
4.2	调度算法设计准则	19
4.3	经典调度算法	19
5	进程同步机制	22
5.1	基本概念	22
5.2	实现方法	22
5.2.1	解决条件和问题综述	22
5.2.2	软件方法	23
5.2.3	软件方法	23
5.2.4	硬件方法	25
5.2.5	信号量 (Semaphores)	27
5.2.6	常见锁类型的实现原理与伪代码	29
5.3	经典同步问题	32
5.3.1	生产者-消费者问题	32
5.3.2	读者-写者问题	33
5.3.3	哲学家就餐问题	40
5.3.4	睡眠理发师问题	41
5.4	进程通信 (IPC) 机制	43
6	死锁	46
6.1	基本概念:	46
6.1.1	定义与产生原因	46
6.1.2	产生死锁的必要条件	46
6.1.3	资源分配图	47
6.2	处理策略	48
6.2.1	死锁预防: 破坏必要条件	48
6.2.2	死锁避免: 银行家算法 (安全序列)	49
6.2.3	死锁检测: 化简资源分配图	53

6.2.4	死锁解除	53
7	内存管理	54
7.1	基础概念	54
7.2	内存分配方案	56
7.2.1	连续分配	56
7.2.2	离散分配	58
7.3	虚拟内存技术	60
7.4	优化技术	63
8	文件系统	63
8.1	文件系统基础	63
8.2	文件组织方式	65
8.2.1	空闲空间管理	65
8.2.2	接口设计	67
8.2.3	连续分配	68
8.2.4	链接分配	69
8.2.5	索引分配	70
8.3	文件目录与路径	71
8.3.1	文件链接	71
8.4	文件操作接口	72
8.5	典型文件系统	72
8.6	文件系统性能	73
8.7	虚拟文件系统 (VFS)	73
9	设备管理	74
9.1	基本概念	74
9.2	硬盘	77
9.2.1	机械硬盘	77
9.2.2	固态硬盘	77
9.2.3	磁盘调度算法	79
9.2.4	分区与格式化	82

9.3	时钟	82
9.4	I/O 系统层次架构	83
9.5	中断处理程序	84
9.5.1	上下文保护与恢复	84
9.5.2	上半部与下半部机制	85
9.6	设备独立内核软件技术	86
9.6.1	缓冲技术	87
9.6.2	异步 I/O (AIO)	89
9.6.3	SPOOLing 技术 (假脱机)	91
9.6.4	设备分配与回收	92

1 操作系统基础知识

1.1 基本概念

操作系统的定义和作用：操作系统是管理硬件资源和为应用程序提供基础服务的核心系统软件

让系统易于使用，且正确高效地运行，是用户与计算机硬件之间的接口，管理硬件资源且对硬件进行抽象

操作系统的四大特征

1. 并发性 (Concurrency): 多个进程或线程可以同时运行
2. 共享性 (Sharing): 多个用户或进程可以共享系统资源
3. 虚拟性 (Virtualization): 提供抽象的资源视图，如虚拟内存、虚拟文件系统
4. 异步性 (Asynchrony): 事件和操作可以在不同时间发生，系统需要处理异步事件

操作系统的四大功能

1. 进程管理
2. 内存管理
3. 文件系统管理
4. 设备管理

1.2 操作系统的历史演化

1. 无操作系统，手动管理硬件资源, 用户独占计算机资源，资源利用率低

2. 单道批处理系统，引入作业调度和批处理

批处理技术：是指计算机系统对一批作业自动进行处理的一种技术。用洗衣店举例，早期没有批处理的时候，你要手动伺候每一台洗衣机，而批处理就理解为给洗衣店安装了一个自动化系统，顾客把衣服送到店里，系统会自动把衣服分类、安排洗衣机工作、烘干、折叠，最后通知顾客来取衣服。

3. 多道程序设计，让 CPU 在一个程序等待 I/O 时可以继续执行其他程序，从而显著提高了资源利用率
4. 分时系统，它允许多个用户通过终端同时与计算机系统进行交互，每个用户都能以相对独立的方式使用系统资源。操作系统通过将 CPU 的时间分成细小的片段（称为时间片），并快速地在多个用户任务之间切换，给每个用户一种“独占”计算机资源的感觉，从而实现多用户的共享计算机资源。
5. 实时系统：是一种计算机系统，它在特定的时间约束下执行任务，对外部事件作出快速响应。实时系统的核心特征在于其时间敏感性，即系统必须在规定的时间内完成某项任务或对某个事件作出响应，否则将导致系统功能失效或严重后果。实时系统通常用于那些对时间要求严格的应用场景，如工业控制、航空航天、军事系统、医疗设备和通信系统等
6. 现代操作系统：支持多核处理器、移动设备和实时系统

1.3 操作系统分类

按应用场景分：

1. 桌面操作系统, 用户友好、图形界面、多媒体支持, 如 Windows、macOS、Linux
2. 服务器操作系统, 高性能、稳定性、安全性, 如 Windows Server、Linux 发行版（如 Ubuntu Server、CentOS）

3. 移动操作系统, 轻量级、触摸优化, 如 Android、iOS
4. 嵌入式操作系统, 资源受限、实时性强, 如 FreeRTOS、VxWorks

按架构特征分类:

1. 单用户系统, 简化管理, 提高性能, 如 DOS
2. 多用户系统, 资源共享和安全性, 如 UNIX、Linux
3. 单任务系统, 简单高效, 如早期的 DOS
4. 多任务系统, 提高资源利用率, 如 Windows、Linux

按内核结构分类:

什么是操作系统的内核: 负责管理所有的资源, 并为上层应用提供服务。没有内核, 软件就无法指挥硬件干活

1. 宏内核结构, 所有操作系统服务都运行在内核空间, 如传统的 UNIX、Linux
2. 微内核结构, 只保留最基本的功能在内核空间, 如 MINIX、QNX
3. 混合内核结构, 结合宏内核和微内核的优点, 关键服务在内核, 其他在用户态如, Windows NT、macOS

1.4 操作系统结构

操作系统是一个复杂的大型软件, 为了控制其设计的复杂度, 通常采用“策略与机制分离”的原则, 并利用模块化、抽象、分层和层级等方法进行管理。

常见的操作系统内核结构主要包括以下四种:

1. 简单结构 (Simple Structure)

- 代表系统: MS-DOS。

- 特点：这种结构通常出现在早期的操作系统中，结构简单，没有清晰的模块划分。

2. 宏内核结构 (Monolithic Kernel)

- 代表系统：初始的 UNIX、Linux、传统的 Unix。
- 定义：所有的服务都在内核空间运行。系统调用之下和物理硬件之上的部分均为内核，内核通过系统调用提供文件系统、CPU 调度、内存管理和其他功能。
- 优点：运行效率高、通信效率高。
- 缺点：结构难以理解、难以维护；随着系统复杂性的增加，其可靠性和安全性难以保障（稳定性风险高）。

3. 模块化结构 (Modular Structure)

- 代表系统：现代操作系统（如 Unix、Linux、Windows 等）均采用了模块化的策略组织各功能。例如 Solaris 由核心内核及 7 个可加载的内核模块构成。
- 定义：模块功能独立且被封装，具有良好定义的接口。Linux 系统也被描述为“宏内核 + 模块化设计”。
- 优点：易于理解，效率高。
- 缺点：全局函数使用多造成访问控制困难；结构不清晰会导致可理解性、可维护性及可移植性差；存在潜在的性能退化。

4. 微内核结构 (Microkernel)

- 代表系统：Mach、Minix、QNX、L4。
- 定义：基于客户/服务器模型，由微内核和核外用户空间的服务器进程构成。

- 内核功能：微内核仅保留最核心的功能，如进程管理、内存管理和通信功能。
- 通信机制：通过消息传递（Message Passing）使客户程序与服务程序进行通信。
- 优点：
 - 易于扩展。
 - 易于从一个硬件架构移植到另一个。
 - 更可靠（因内核代码更少）。
 - 更安全。
- 缺点：用户空间到内核空间的通信增加了系统开销，早期性能不及宏内核结构。

1.5 操作系统的启动和导引

操作系统的启动是一个分阶段的过程，旨在解决“如何让一台空白的计算机运行起复杂的操作系统”这一核心问题。该过程主要分为三个阶段：

1. 阶段 1：硬件自举 (**Hardware Bootstrapping**) ——解决“CPU 执行起点”问题
 - 核心目标：让上电后“空白”的 CPU 具备基本执行能力，完成硬件自检，并找到下一阶段的程序。
 - **ROM 启动**：CPU 上电后，会固定跳转到主板 ROM（只读存储器）的特定地址（如 RISC-V 的 0x1000）执行固化的初始程序。
 - **BIOS 任务**：
 - (a) 检查硬件：检测内存、硬盘等硬件是否正常。
 - (b) 初始化硬件：启动内存控制器、显示器和 I/O 设备。
 - (c) 加载 **Bootloader**：从存储设备（如硬盘引导扇区）读取 Bootloader 到内存，并移交 CPU 控制权。

2. 阶段 2：引导加载器 (Bootloader) ——搭建“硬件与内核”的桥梁

- 核心目标：作为中间层，解决 BIOS 功能简单无法直接加载复杂内核的问题，为内核运行准备环境。
- 环境准备：隐藏硬件差异，设置内存布局，初始化栈空间。
- 加载内核：Bootloader 识别文件系统（如 ext4, FAT32），将操作系统内核文件（如 vmlinuz）从硬盘读入内存指定区域。
- 移交控制：将收集到的硬件配置信息传递给内核，并将 CPU 控制权正式移交给操作系统内核。

3. 阶段 3：内核初始化 (Kernel Initialization) ——建立“完整的操作系统环境”

- 核心目标：从单程序执行过渡到多任务系统，建立资源管理体系。
- 搭建管理体系：
 - (a) 内存管理：建立页表，启用 MMU，实现虚实地址映射。
 - (b) 进程管理：初始化进程调度队列（就绪/阻塞队列），支持并发。
 - (c) 文件系统：挂载根文件系统。
- 加载驱动与中断：加载网卡、显卡等驱动，建立中断向量表。
- 启动用户环境：启动第一个用户态进程（如 Linux 的 init/systemd），加载登录界面，等待用户登录。

1.6 应用程序的执行流程

1. 加载：当启动一个程序时，由 OS 的加载器将该程序的可执行文件加载到内存中。
2. 运行：OS 创建一个新的进程，并将 CPU 的控制权交给该进程。该新进程开始执行加载到内存中的程序代码。
3. 系统调用：当程序需要操作系统提供的服务（如文件操作、内存分配等）时，会通过系统调用接口向操作系统发出请求。

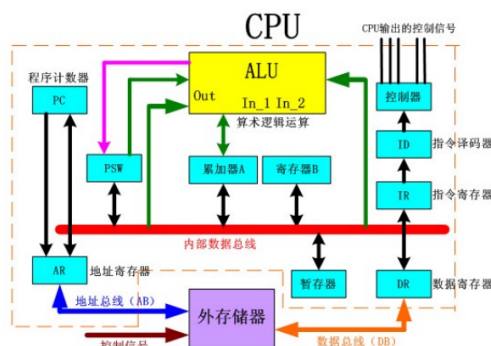


图 1: CPU 重要寄存器示意图

4. 中断处理：如果程序执行过程中发生了中断（如 I/O 完成、定时器中断等），CPU 会暂停当前进程的执行，转而执行操作系统的中断处理程序。
5. 退出：当程序执行完成或者由于某种原因需要停止时，它将执行退出操作，包括释放资源，关闭打开的文件，通知父进程等。

2 操作系统运行环境与运行机制

2.1 硬件支持

CPU 重要寄存器的作用：

1. 程序计数器 (**Program Counter, PC**)：存储下一条将要执行的指令的地址。
2. 指令寄存器 (**Instruction Register, IR**)：IR 寄存器用于记录最近取出并解码的指令。
3. 程序状态字 (**PSW**) 寄存器：PSW 寄存器用于记录处理器的运行状态，如条件码、模式、控制位等。PSW 寄存器中的一些位可以影响处理器的行为，如中断允许位、处理器状态位等
4. 通用寄存器 (**General Purpose Registers**)：用于存储临时数据和计算结果。

5. 控制寄存器 (**Control Registers**): 控制寄存器是一组只能被操作系统程序使用的寄存器, 用于控制和配置处理器的一些特性和功能。比如, 在多任务环境下 (像电脑同时开着浏览器、微信), 控制寄存器可以设置 CPU 的”工作模式”, 区分普通程序 (用户模式) 和系统程序 (内核模式), 确保普通程序不能乱改系统的关键设置。

运行模式:

表 1: 内核态与用户态对比

对比维度	内核态 (Kernel Mode)	用户态 (User Mode)
程序类型	操作系统的程序执行	用户程序执行
指令权限	使用全部指令	禁止使用特权指令
资源访问	使用全部系统资源 (包括整个存储区域)	只允许用户程序访问自己的存储区域

CPU 特权级切换

1. 应用程序调用操作系统提供的系统调用
2. 应用程序执行一条指令触发了异常
3. 应用程序执行过程中, CPU 收到一条来自外设的中断

2.2 中断与异常

因为中断处理程序运行在核心态 (操作系统的特权模式), 而原程序通常运行在用户态 (普通程序的非特权模式), 所以需要硬件支持来完成 “用户态 → 核心态” 的切换。

概念:

- 中断: 来自 CPU 外部的的事件, 如 I/O 设备请求服务
- 异常: 来自 CPU 内部的事件, 如除零错误、非法指令等

中断向量表:

它是一个存储在内存固定区域的数据结构, 表中的每一项存放的是中断处理程序的入口地址 (也就是程序计数器 PC 的值) 和状态字 (PSW)

中断处理过程

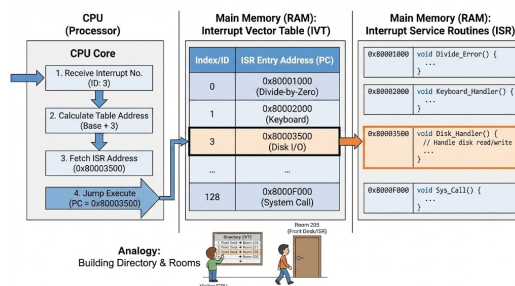


图 2: 中断向量表示意图

- 保存现场：保存当前进程的状态，包括程序计数器（PC）、程序状态字（PSW）和其他寄存器的内容，以便中断处理完成后能够恢复执行。
- 恢复现场：从保存的状态中恢复进程的执行状态，包括恢复程序计数器（PC）、程序状态字（PSW）和其他寄存器的内容。
- 中断服务程序执行：根据中断向量表找到对应的中断处理程序，并执行相应的处理逻辑。

2.3 系统调用

是操作系统提供的服务的编程接口，通常用高级语言（如 C 或 C++）编写，大多数程序通过高级应用程序编程接口（API）而不是直接使用系统调用，最常见的三个 API：Windows 的 Win32 API，基于 POSIX 的系统（包括几乎所有版本的 UNIX、Linux 和 Mac OS X）的 POSIX API，以及 Java 虚拟机（JVM）的 Java API

作用：用户态请求内核服务，用户程序与操作系统之间的桥梁
实现机制：

1. 访管指令 (Trap Instruction): 用户程序执行一条特殊的指令（如 x86 的 int 指令），触发软中断，切换到内核态
2. 系统调用表 (System Call Table): 内核维护一个系统调用表，存储各个系统调用对应的处理程序地址
3. 系统调用号 (System Call Number): 用户程序通过寄存器或栈传递系统调用号，内核根据该号查找系统调用表

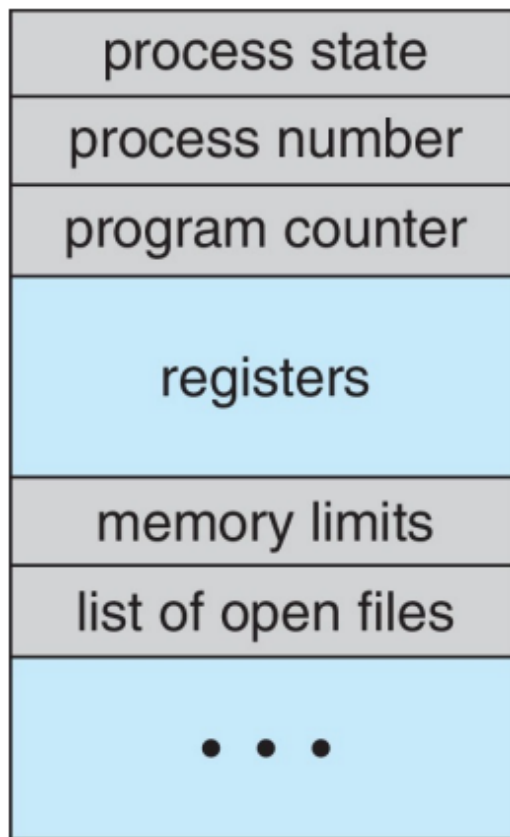


图 3: 进程控制块 PCB 示意图

3 进程/线程模型

3.1 进程 (Process)

3.1.1 相关概念

定义：正在执行的程序，具有动态性、生命周期（从创建到消亡），执行时需要处理机资源（资源分配的基本单位，程序的一次执行）

进程控制块 **PCB**：管理程序运行的数据结构

PCB 主要内容：

- 进程状态：运行、等待、就绪等
- 进程标识符：唯一标识进程的 ID

- 进程队列指针：用于记录 PCB 队列中下一个 PCB 的地址
- 程序和数据地址：进程的程序和数据在内存或外存中的存放地址
- 进程优先级：反映进程获得 CPU 的优先级别
- **CPU 现场保护区**：CCPU 现场信息的存放区域，包括：通用寄存器、程序计数器、程序状态字等
- 通信信息：进程与其他进程所发生的信息交换时所记录的有关信息
- 家族关系：指明本进程与家族的关系，如父子进程标识
- 资源信息：进程所拥有的资源及其使用情况

程序转换为进程的过程：

1. 程序加载：操作系统将可执行文件加载到内存中，分配必要的资源
2. 创建 **PCB**：为新进程创建进程控制块，初始化各项信息
3. 设置初始状态：将进程状态设置为就绪，等待调度
4. 调度执行：操作系统调度进程进入运行状态，开始执行指令

3.1.2 状态变迁

进程的生命周期包含一系列状态的转换。最基本的模型包含五个状态：新建（New）、就绪（Ready）、运行（Running）、等待（Waiting）和终止（Terminated）。基本的主干线路为：新建 → 就绪 → 运行 → 终止。

1. 基本状态转换详解：

- 就绪 → 运行 (**Scheduler Dispatch**): 当调度程序选择该进程执行时，进程获得 CPU 资源。
- 运行 → 就绪 (**Interrupt**):
 - 时间片用完：进程分配的 CPU 时间片耗尽，必须让出 CPU。
 - 被抢占：有更高优先级的进程进入就绪队列。

- **新建 New**: 进程被创建
- **运行 Running**: 指令执行时的状态
- **等待 Waiting**: 进程等待某些事件发生时的状态
- **就绪 Ready**: 进程等待获得 CPU
- **终止 Terminated**: 进程正常或异常结束时的状态



图 4: 进程五状态变迁图示意图

- 运行 → 等待 (**I/O or Event Wait**): 这是一个主动的过程。进程请求 I/O 操作（如读写磁盘）或等待某事件（如等待子进程结束），暂时无法继续执行。
- 等待 → 就绪 (**I/O or Event Completion**): 这是一个被动的过程。当 I/O 完成或等待的事件发生（如接收到信号），进程被唤醒，重新进入就绪队列等待调度。
- 运行 → 终止 (**Exit**): 进程执行完毕或遇到错误退出。

2. 挂起状态 (Suspend) 的引入:

为了解决内存紧张或用户干预等问题，引入了“挂起”状态。在挂起状态下，进程会被保存在磁盘（外存）上，不占用内存资源，也不占用 CPU 时间。

3. 扩展后的状态转换（七状态模型）：引入挂起后，增加了挂起就绪 (**Suspend Ready**) 和挂起阻塞/等待 (**Suspend Waiting**) 两个新状态：

- 挂起 (**Suspend**):

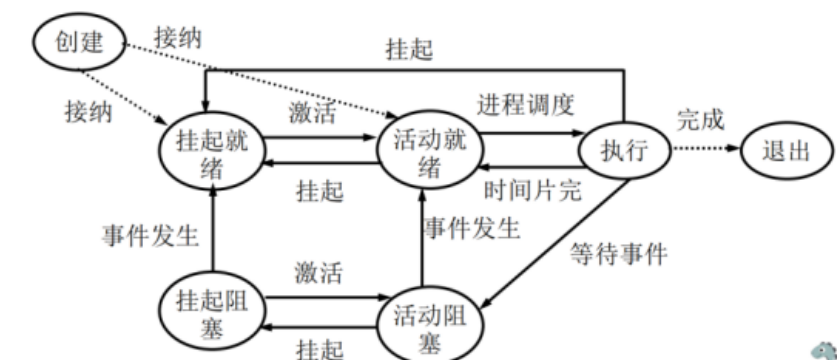


图 5: 引入挂起状态后的进程状态扩展图

- 活动就绪 → 挂起就绪: 当内存不足或系统负荷过高时, 系统将就绪进程换出到磁盘。
- 活动阻塞 → 挂起阻塞: 当进程处于等待状态且内存不足时, 将其换出。

- 激活 (Activate):

- 挂起就绪 → 活动就绪: 内存有空闲或该进程优先级较高时, 将其调入内存。
- 挂起阻塞 → 活动阻塞: 同样因内存可用而被调入, 但仍需等待事件。

- 挂起状态下的事件发生:

- 挂起阻塞 → 挂起就绪: 进程在外存中等待的事件发生了 (如 I/O 结束), 虽然它还在磁盘上, 但状态变为 “就绪”, 一旦被激活即可运行。

3.1.3 进程控制

进程控制是操作系统内核通过原语 (Primitive) 和系统调用 (System Call) 对进程生命周期进行管理的核心机制。主要包含以下四个方面:

1. 进程创建 (Creation) —— `fork()` 与 `exec()`

- 机制：操作系统创建一个具有唯一标识符（PID）的新进程，并为其分配 PCB 结构。
- **fork()** 系统调用：
 - 用于创建一个新进程（子进程）。
 - 子进程是父进程的副本（复制地址空间），但拥有独立的 PID。
 - 返回值：在父进程中返回子进程的 PID (> 0)，在子进程中返回 0。
- **exec()** 系统调用：
 - 通常在 **fork()** 之后使用。
 - 用于将当前进程的内存空间替换为新的程序代码并开始执行，从而让子进程运行不同的任务。

2. 进程撤销/终止 (Termination) —— **exit()**

- 触发条件：进程完成任务正常退出，或因错误（如非法内存访问）被强制退出。
- 处理流程：
 - 进程执行 **exit()** 系统调用请求删除自身。
 - 操作系统回收该进程占用的资源（内存、文件等）归还给父进程或系统。
 - 撤销该进程的 PCB，将其从总链队列中摘除。

3. 进程阻塞 (Blocking) —— **wait()**

- 定义：进程因等待某事件发生（如 I/O 完成、资源可用）而暂停执行，主动放弃 CPU。
- **wait()** 系统调用：
 - 父进程调用 **wait()** 以等待子进程终止。
 - 如果子进程尚未结束，父进程会进入阻塞状态，直到接收到子进程的退出信号。
 - 该调用还可以获取子进程的退出状态码。

- 内部动作：保存当前 CPU 现场到 PCB，将进程状态置为“等待/阻塞”，并将 PCB 插入等待队列，转而调度其他进程。

4. 进程唤醒 (Waking up)

- 定义：当进程等待的事件发生时（如 I/O 结束、子进程退出），由发现者（如中断处理程序或另一进程）将其唤醒。
- 处理流程：
 - 在等待队列中找到该进程的 PCB。
 - 将其移出等待队列。
 - 将进程状态修改为“就绪” (Ready)。
 - 将 PCB 插入就绪队列，等待调度程序分配 CPU。

3.2 线程

3.2.1 相关概念

定义：进程内的一个执行单元，是 CPU 调度和分派的基本单位。线程是比进程更小的独立运行单位，一个进程可以包含多个线程。

为什么要引入线程：

1. 多线程应用的瓶颈问题
2. 应用程序中的多个任务可以通过多个线程来实现
3. 线程的创建是轻量级的，而进程的创建是重量级的
4. 使用多线程可以简化代码，提高效率
5. 内核通常也是多线程的

多线程的优势：

1. 响应性：进程在某个部分被阻塞的情况下被允许继续执行，对于用户界面尤其重要

2. 资源共享：同一进程的多个线程共享进程的资源，如内存空间和文件描述符
3. 经济性：创建和销毁线程的开销远小于进程
4. 可扩展性：进程可以利用多核架构的优势

3.2.2 线程与进程的关系

- 线程是属于进程的，它们可以被看作是在进程内部的一个个独立的执行路径
- 线程共享进程的资源，如内存空间和文件描述符
- 线程间的通信比进程间的通信更简单，线程的切换开销也比进程切换要小

3.2.3 线程的实现方式：

1. 用户级线程 (User-Level Threads, ULT)：

- 线程管理完全在用户空间进行，操作系统内核并不感知线程的存在
- 优点：线程切换开销小，创建和销毁速度快
- 缺点：无法利用多核处理器，阻塞一个线程会阻塞整个进程

2. 内核级线程 (Kernel-Level Threads, KLT)：

- 线程由操作系统内核管理，内核知道每个线程的存在
- 优点：可以利用多核处理器，一个线程阻塞不会影响其他线程
- 缺点：线程切换开销较大，创建和销毁速度较慢

3. 混合线程模型 (Hybrid Thread Model)：

- 结合了用户级线程和内核级线程的优点

- 用户级线程映射到内核级线程上，允许多个用户级线程共享一个内核级线程
- 优点：既能利用多核处理器，又能减少线程切换开销

简单理解方式：

- 用户级线程 → 门经理（用户态线程库）自己管理，不告诉总部（内核）
- 内核级线程 → 总部（内核）直接管理每个小组（线程）
- 混合线程模型 → 门经理（用户态线程库）和总部（内核）共同管理

4 进程/线程调度

4.1 基本概念

调度：协调每个请求对资源的使用的方法

多级调度：分为长程调度（作业调度）、中程调度（交换调度）和短程调度（CPU 调度）

- 长程调度 (Long-Term Scheduling)：它决定了哪个程序可以被操作系统接纳，为其创建进程并放入就绪队列；控制多道程序的数量
- 中程调度 (Medium-Term Scheduling)：它负责将部分进程从内存中换出到外存（挂起），以释放内存空间给其他进程使用；当内存资源充足时，再将这些进程换入内存（激活）；避免内存使用过多
- 短程调度 (Short-Term Scheduling)：它决定了哪个进程可以获得 CPU 使用权，进行实际的指令执行，实现多任务的并发执行；决定进程的执行权

抢占式调度和非抢占式调度

- 抢占式调度 (Preemptive Scheduling)：操作系统可以在任何时候中断正在运行的进程，将 CPU 分配给另一个进程。这种方式允许更高优先级的进程获得 CPU 使用权，提高系统响应速度。

- 非抢占式调度 (Non-Preemptive Scheduling): 一旦进程获得了 CPU 使用权, 它将一直运行直到它自愿放弃 CPU (如进入等待状态或终止)。这种方式简单, 但可能导致低优先级进程长时间得不到执行。

上下文切换: 当操作系统从一个进程切换到另一个进程时, 需要保存当前进程的状态 (上下文) 并加载新进程的状态, 这个过程称为上下文切换。上下文切换包括保存和恢复寄存器状态、程序计数器、内存映射等信息。

4.2 调度算法设计准则

- CPU 利用率 (CPU Utilization): 量 CPU 被有效利用的程度的指标。理想情况下, 希望 CPU 能够一直被占用
- 吞吐量 (Throughput): 单位时间内完成的进程数量。提高吞吐量意味着系统能够更高效地处理任务
- 周转时间: 任务提交到任务完成所经过的时间, 包括等待时间和执行时间

$$\text{周转时间} = \text{等待时间} + \text{执行时间} = \frac{\text{总等待时间}}{\text{任务数}} + \frac{\text{总执行时间}}{\text{任务数}}$$
- 等待时间: 衡量任务在就绪队列中等待的总时间
- 响应时间: 任务提交到首次开始执行所经过的时间

4.3 经典调度算法

- 先来先服务 (FCFS): 先来先执行
- 短作业优先 (SJF) (非抢占): 将进程和后续进程的 CPU 执行时间进行比较, 选择调度 CPU 执行时间最短的进程
- 最短剩余时间优先 (SRTN) (SJF 抢占版): 考虑进程的到达时间, 在新进程到达进行更新选择剩余执行时间最短的进程
- 最高响应比优先 (HRRN) (非抢占): 响应比 = (等待时间 + 服务时间) / 服务时间, 选择响应比最高的进程

- 考虑进程的到达时间，且允许抢占

Process	ArrivalTime	Burst Time
P ₁	0	8
P ₂	1	4
P ₃	2	9
P ₄	3	5

- 抢占式 SJF 甘特图

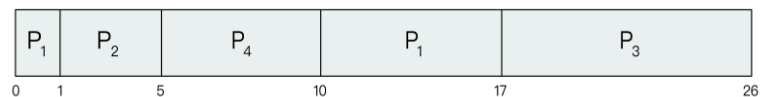
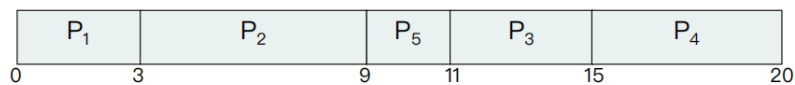


图 6: 最短剩余时间优先 (SRTN) 调度示意图

Process	ArrivalTime	Burst Time
P ₁	0	3
P ₂	1	6
P ₃	2	4
P ₄	3	5
P ₅	4	2

- 甘特图



- 0: P₁执行, P₂ P₃ P₄依次到达;
- 3: $r_2 = 1 + 2/6$, $r_3 = 1 + 1/4$, $r_4 = 1$; P₂先运行;
- 9: $r_3 = 1 + 7/4$, $r_4 = 1 + 6/5$, $r_5 = 1 + 5/2$; P₂先运行;
- 11: $r_3 = 1 + 9/4$, $r_4 = 1 + 8/5$; P₃先运行;

图 7: 最高响应比优先 (HRRN) 调度示意图

- 时间片轮转 (RR): 为每个进程分配一个固定的时间片，时间片用完后将进程放回就绪队列末尾
- 优先级调度: 为每个进程分配一个优先级，优先级高的进程优先执行，可以是抢占式或非抢占式

多级队列：将进程根据某些特征（如优先级、类型）划分到不同的队列中，每个队列有自己的调度算法

多级反馈队列：结合了时间片轮转和优先级调度，允许进程在不同优先级队列间动态迁移

表 2: 多级队列与多级反馈队列调度算法对比

对比维度	多级队列调度算法	多级反馈队列调度算法
进程分类方式	静态分类：进程一旦入队，终身属于该队列	动态分类：进程会根据执行情况在队列间迁移
队列间迁移	无迁移：进程入队后不会改变所属队列	有迁移：若进程时间片耗尽，可能降入低优先级队列；若进程长期未被调度，可能升入高优先级队列（防止饥饿）
灵活性	低：队列划分固定，无法适应进程执行的动态变化	高：动态调整队列，更贴合实际运行场景
饥饿问题	易出现：低优先级队列可能长期得不到调度	不易出现：通过队列迁移机制避免“饥饿”
实现复杂度	较低：只需按类型划分队列并配置调度算法	较高：需设计队列迁移规则、优先级调整逻辑等
典型应用场景	进程类型明确且稳定的系统（如专用服务器的批处理 + 交互任务分离）	通用操作系统（如 Linux、Windows），需兼顾多种任务类型的调度

多处理器调度的亲和性：指进程倾向在特定处理器或处理器集合上运行的特性

原因：由于该进程或线程之前在这些处理器上运行过因此可能有一些数据或状态仍然缓存在那里，如果再次在这些处理器上运行，则可以利用这些缓存数据，从而提高性能。

5 进程同步机制

5.1 基本概念

竞态条件：多个进程/线程同时进入临界区，执行结果与访问发生的特定顺序有关

临界区：访问共享资源的一段代码

原子操作：原子操作是实现操作系统原语 (Primitive) 的基础，一个操作（或一系列操作）在执行过程中，绝对不会被任何机制（如中断、信号、其他线程）打断

原子：不可分割的最小单位

5.2 实现方法

5.2.1 解决条件和问题综述

解决临界区问题需要满足的几个条件：

- 互斥：同一时刻只能有一个进程/线程进入临界区
- 进展性：如果没有进程在临界区内，且有进程想进入临界区，那么必须有一个进程能够进入
- 有界等待：一个进程发出进入临界区的请求后，其他进程被允许进入其临界区的次数必须是有限的。

两种并发问题：

- 是共享资源引起
- 是占有 cpu 导致饥饿问题

5.2.2 软件方法

软件方法是指仅通过代码逻辑（变量标记、循环判断）来实现互斥，而不依赖特殊的硬件指令。

5.2.3 软件方法

软件方法是指仅通过代码逻辑（变量标记、循环判断）来实现互斥，而不依赖特殊的硬件指令。

1. 单标志法 (Strict Alternation)

- 机制：设置一个公用整型变量 `turn`，用于指示允许进入临界区的进程编号。
- 核心逻辑：

```
// 进程 P0                // 进程 P1
while (turn != 0);         while (turn != 1);
// 临界区                  // 临界区
turn = 1;                  turn = 0;
```

- 缺点：违背了“进展性”。如果进程 0 不想进入临界区，它不会把 `turn` 改为 1，导致进程 1 即使想进也无法进入，资源闲置。

2. 双标志法先检查 (Check then Set)

- 机制：设置一个布尔数组 `flag[]`。进程先检查对方是否想进，若不想，则设置自己标志并进入。
- 核心逻辑：

```
while (flag[j]); // 1. 先检查对方是否想进
flag[i] = true;  // 2. 后设置自己想进
// 临界区
flag[i] = false; // 3. 退出时修改标志
```

- 缺点：违背了“互斥”。如果两个进程同时检测到 `flag` 为 `false`，会同时运行到第 2 行设置标志，从而同时进入临界区。

3. 双标志法后检查 (Set then Check)

- 机制：先设置自己的标志 `flag[i]=true`，再检测对方。
- 核心逻辑：

```
flag[i] = true;    // 1. 先设置自己想进
while (flag[j]);   // 2. 后检查对方是否想进
// 临界区
flag[i] = false;   // 3. 退出
```

- 缺点：可能导致“死锁”。如果两个进程同时举旗（执行第 1 行），然后在第 2 行互相盯着对方，谁也进不去，导致无限等待。

4. 皮特森算法 (Peterson's Algorithm)

- 机制：结合了双标志法和单标志法。引入 `flag[]` 表示意愿，引入 `turn` 表示谦让。
- 核心逻辑：

```
flag[i] = true; // 1. 我想进（意愿）
turn = j;       // 2. 让你先进（谦让）
// 3. 检查：如果你想进 且 确实轮到你，那我就等
while (flag[j] && turn == j);
// 临界区
...
flag[i] = false; // 4. 退出
```

- 评价：完美解决了互斥、进展性和有界等待问题，是经典的软件解决方案。

5.2.4 硬件方法

硬件方法是操作系统中解决互斥问题（临界区问题）的另一种重要途径。与软件方法试图通过复杂的逻辑判断来避免冲突不同，硬件方法直接利用 CPU 提供的特殊指令，从底层保证某些操作的原子性。

主要有以下三种经典的硬件方法：

1. 关中断 (Disable Interrupts)

- 原理：CPU 进行进程切换通常是由中断（如时钟中断）触发的。如果一个进程在进入临界区之前关闭中断，CPU 就听不到外界的任何信号，也就不会发生进程切换，从而独占 CPU。
- 伪代码：

```
disable_interrupts(); // 关中断
// ... 临界区代码 ...
enable_interrupts(); // 开中断
```

- 缺点：不适用于多处理器系统；将关中断权限交给用户进程极其危险；限制了 CPU 处理其他紧急任务的能力。

2. Test-and-Set (TSL) 指令

- 机制：硬件保证读取旧值和设置新值这两个动作是原子的（不可分割）。如果锁是 FALSE，读取并设置为 TRUE（抢锁成功）；如果锁是 TRUE，读取并设置为 TRUE（锁被占）。
- 核心逻辑：

```
// 硬件原子指令定义
boolean TestAndSet(boolean *target) {
    boolean rv = *target; // 1. 读旧值
    *target = TRUE;       // 2. 设新值（上锁）
    return rv;            // 3. 返回旧值
}
```

```

// 进程逻辑
while (TestAndSet(&lock)); // 自旋等待：只要返回 true 就一直循环
// ... 临界区 ...
lock = FALSE;                // 解锁

```

- 特点：适用于多处理器，但会导致忙等待 (**Busy Waiting**)，浪费 CPU 时间。

3. Swap (Exchange) 指令

- 机制：原子地交换两个内存位置的值。进程利用手中的 **key**（初始为 true）与全局的 **lock** 进行交换，直到换回来 false 为止。
- 核心逻辑：

```

// 硬件原子指令定义
void Swap(boolean *a, boolean *b) {
    boolean temp = *a;
    *a = *b;
    *b = temp;
}

// 进程逻辑
key = TRUE;
while (key == TRUE) {
    Swap(&lock, &key); // 尝试交换
}
// ... 临界区 ...
lock = FALSE;          // 解锁

```

- 特点：简单易实现，适用于多处理器，但同样存在忙等待问题，可能导致饥饿。

5.2.5 信号量 (Semaphores)

信号量 S 是包含一个整数值的对象，只能通过两个原子操作 `wait()` 和 `signal()` 访问，有时也称为 $P()$ 和 $V()$ 操作。

可以把信号量理解为只通过原子操作控制的整形变量。

```
// P操作 (Wait)                                // V操作 (Signal)
wait(S) {                                        signal(S) {
    while (S <= 0); // 自旋等待(或阻塞)          S = S + 1;    // 释放资源
    S = S - 1;    // 占用资源                    }
}
```

信号量机制在操作系统中主要有两大应用场景：实现互斥和实现同步。

1. 利用信号量实现互斥 (Mutual Exclusion)

- 原理：将信号量 S 初始化为 1（代表只有 1 个资源或一把锁）。
- 流程：
 - 在进入临界区之前执行 $P(S)$ ：若 $S = 1$ ，则 S 变为 0，进程进入；若 $S = 0$ ，进程等待。
 - 在退出临界区之后执行 $V(S)$ ：将 S 恢复为 1，唤醒其他等待的进程。
- 伪代码结构：

```
semaphore mutex = 1; // 初始化互斥信号量为 1

P(mutex);           // 1. 关门：申请资源
// -----
//   临界区 (Critical Section)
//   (读写共享数据)
// -----
V(mutex);           // 2. 开门：释放资源
```

2. 利用信号量实现同步 (Synchronization)

- 原理：将信号量 S 初始化为 0（代表资源初始不可用，或操作尚未完成）。
- 目标：保证进程 P_1 的语句 x 执行完后，进程 P_2 才能执行语句 y （即前趋关系 $x \rightarrow y$ ）。
- 流程：
 - 前操作进程 (P_1)：执行完任务后，执行 $V(S)$ ，将 S 变为 1，通知对方“我做完了”。
 - 后操作进程 (P_2)：执行前先执行 $P(S)$ 。若 P_1 还没做完 ($S = 0$)， P_2 会阻塞等待；只有当 P_1 执行了 $V(S)$ 后， P_2 才能通过 $P(S)$ 继续执行。
- 伪代码结构：

```
semaphore S = 0;           // 初始化同步信号量为 0

// 进程 P1（先执行）           // 进程 P2（后执行）
{                               {
    代码 x;                     P(S); // 检查P1是否完成?
    V(S); // 通知P2             代码 y;
}                               }
```

同步：强制让两个原本各跑各的进程，按照我们规定的先后顺序来执行，为的就是能让后面的进程拿到前面进程的数据，而不是后面的进程执行完才发现前面的进程还没执行完，数据还没准备好 - 总结：

- 互斥： S 初值为 1， P/V 操作在同一个进程中成对出现（一前一后夹住临界区）。
- 同步： S 初值为 0， P/V 操作在两个不同的进程中成对出现（一个发信号，一个等信号）。

5.2.6 常见锁类型的实现原理与伪代码

根据线程等待锁的方式（忙等待 vs 睡眠等待）及控制粒度的不同，锁的内部实现逻辑存在显著差异。

1. 自旋锁 (Spinlock)

- 机制：当线程尝试获取锁失败时，它不会挂起（睡眠），而是通过无限循环（忙等待）的方式不断检查锁是否变为空闲。
- 底层原理：通常基于硬件的 Test-and-Set (TSL) 或 Compare-and-Swap (CAS) 原子指令实现。
- 优点：没有进程上下文切换（Context Switch）的开销，获取锁的速度极快。
- 缺点：如果持有锁的线程长时间不释放，等待的线程会一直空转，白白浪费 CPU 周期。
- 适用场景：多核处理器，且临界区极短（执行时间短于上下文切换时间）的情况。
- 伪代码实现：

```
// lock_flag: 0为空闲, 1为占用
void spin_lock(int *lock_flag) {
    // 原子操作：读取旧值并设为1
    // 若返回1(原已占用)，则条件为真，继续循环(忙等待)
    // 若返回0(原空闲)，则条件为假，循环结束(且已设为1)
    while (TestAndSet(lock_flag) == 1) {
        ; // 空转，消耗时间片
    }
}

void spin_unlock(int *lock_flag) {
    *lock_flag = 0; // 简单重置为空闲
}
```


2. 互斥锁 (Mutex)

- 机制：当线程尝试获取锁失败时，它会主动放弃 **CPU**，进入阻塞 (Sleep) 状态，并加入等待队列。当锁被释放时，操作系统会唤醒等待的线程。
- 优点：不会忙等待，节省 CPU 资源，适合长时间持有锁的场景。
- 缺点：涉及“睡眠”和“唤醒”，需要操作系统介入进行上下文切换，开销相对自旋锁较大。
- 适用场景：临界区较长，或者无法预测持有锁时间的情况
- 伪代码实现：

```
void mutex_lock(Mutex *m) {
    // 尝试原子加锁
    if (TestAndSet(&m->flag) == 1) {
        // 失败：加入队列并睡眠
        add_to_queue(m->wait_queue, current_thread);
        block(); // 关键点：发生上下文切换
    }
}

void mutex_unlock(Mutex *m) {
    if (is_empty(m->wait_queue)) {
        m->flag = 0; // 没人排队，直接释放
    } else {
        // 有人排队：唤醒一个线程(移交锁的使用权)
        Thread t = remove_from_queue(m->wait_queue);
        wakeup(t); // 关键点：唤醒睡眠的线程
    }
}
```

3. 读写锁 (RWLock)

- 机制：将访问者区分为“读者”和“写者”，遵循以下规则：

- 读-读不互斥：允许多个读者同时进入临界区（共享模式）。
- 读-写互斥：当有人在写时，禁止读取；当有人在读时，禁止写入。
- 写-写互斥：同一时刻只能有一个写者。

• 伪代码实现：

```

Semaphore rw_mutex = 1;    // 保护临界区的写锁
Semaphore count_mutex = 1; // 保护计数器的锁
int readers = 0;           // 正在读的人数

// --- 读者逻辑 ---
void read_lock() {
    P(count_mutex);        // 保护计数器
    readers++;
    if (readers == 1) {
        P(rw_mutex);       // 第一个读者负责锁门！
                           // 此时写者无法进入
    }
    V(count_mutex);
}

void read_unlock() {
    P(count_mutex);
    readers--;
    if (readers == 0) {
        V(rw_mutex);       // 最后一个读者负责开门！
    }
    V(count_mutex);
}

// --- 写者逻辑 ---
void write_lock() {

```

```

        P(rw_mutex);           // 写者必须等待rw_mutex全空闲
    }

```

条件变量：操作系统中一种用于“等待”的同步机制，通常与互斥锁结合使用。条件变量允许线程在某个条件不满足时进入等待状态，并在条件满足时被唤醒继续执行。本质上，条件变量是一种高级同步原语，建立在信号量或互斥锁之上。

5.3 经典同步问题

5.3.1 生产者-消费者问题

问题简述：也称为有界缓冲区问题，假设有一个或多个生产者线程和一个或多个消费者线程。生产者生产数据放入一个有限大小的缓冲区，消费者从缓冲区取出数据进行消费。需要确保生产者不会在缓冲区满时继续生产，消费者不会在缓冲区空时继续消费。

解决要点：解决“生产者生产数据”与“消费者消费数据”的协同问题——既要保证生产者不会往满缓冲区写数据、消费者不会从空缓冲区读数据（同步）

解决方案：

信号量定义：

- semaphore mutex = 1; // 互斥信号量，保护缓冲区
- semaphore empty = n; // 空闲缓冲区数量
- semaphore full = 0; // 已占用缓冲区数量

生产者进程：

```

producer() {
    while(true) {
        produce_item();    // 生产数据
        P(empty);          // 等待空闲缓冲区
        P(mutex);          // 进入临界区
    }
}

```

```

        put_item();          // 放入缓冲区
        V(mutex);           // 离开临界区
        V(full);             // 增加已占用缓冲区数量
    }
}

```

消费者进程:

```

consumer() {
    while(true) {
        P(full);             // 等待已占用缓冲区
        P(mutex);           // 进入临界区
        get_item();          // 从缓冲区取出
        V(mutex);           // 离开临界区
        V(empty);            // 增加空闲缓冲区数量
        consume_item();      // 消费数据
    }
}

```

关键点:

- P(empty) 和 P(full) 必须在 P(mutex) 之前, 否则可能死锁
- V(mutex) 必须在 V(empty)/V(full) 之前, 尽快释放互斥锁

5.3.2 读者-写者问题

问题简述: 多个读者和写者进程访问共享数据。读者可以同时访问数据, 而写者需要独占访问。

解决要点: 我们分三种思路解决该问题, 分别是“读者优先”、“写者优先”和“公平策略”。

先来介绍“读者优先”策略:

解决方案 (读者优先):

我们已经在前面的“读写锁”部分介绍过读者优先的实现方式, 这里再简单回顾一下:

信号量定义:

- semaphore rw_mutex = 1; // 保护临界区的写锁
- semaphore count_mutex = 1; // 保护计数器的锁
- int readers = 0; // 正在读的人数

读者逻辑:

```
void read_lock() {
    P(count_mutex);
    readers++;
    if (readers == 1) {
        P(rw_mutex);    // 第一个读者负责锁门
    }
    V(count_mutex);
}

void read_unlock() {
    P(count_mutex);
    readers--;
    if (readers == 0) {
        V(rw_mutex);    // 最后一个读者负责开门
    }
    V(count_mutex);
}
```

写者逻辑:

```
void write_lock() {
    P(rw_mutex);
}

void write_unlock() {
```

```

    V(rw_mutex);
}

```

存在问题：读者优先可能导致写者”饥饿”——只要不断有读者进入，写者就永远无法获得锁。

解决方案（写者优先）：

为了解决”写者饥饿”问题，我们需要让写者拥有更高的优先权。核心思想是：一旦有写者在等待，就不再允许新的读者进入。

信号量定义：

- semaphore reader_count_lock = 1; // 保护 reader_count 的互斥锁
- semaphore writer_count_lock = 1; // 保护 writer_count 的互斥锁
- semaphore read_try = 1; // 写者优先的”控制门”
- semaphore write_lock = 1; // 核心互斥锁（读写互斥、写写互斥）
- int reader_count = 0; // 当前读者数量
- int writer_count = 0; // 当前等待/执行的写者数量

读者逻辑：

```

void reader() {
    P(read_try);           // 检查是否有写者在等
    P(reader_count_lock);
    reader_count++;
    if (reader_count == 1) {
        P(write_lock);    // 第一个读者锁住写操作
    }
    V(reader_count_lock);
    V(read_try);          // 释放
}

```

表 3: 写者优先方案的信号量组件说明

组件名称	作用
reader_count	记录当前有多少个读者在读取数据（需互斥保护）
writer_count	记录当前有多少个写者在等待或执行写操作（需互斥保护）
reader_count_lock	互斥锁，保护 reader_count 的修改（避免多个读者同时修改计数）
writer_count_lock	互斥锁，保护 writer_count 的修改（避免多个写者同时修改计数）
read_try	写者优先的”控制门”：写者可通过它阻塞新读者进入，确保写者优先执行
write_lock	核心互斥锁，实现”读-写互斥””写-写互斥”（读者和写者、写者和写者不能同时访问数据）

```

    read_data();                // 读取数据

    P(reader_count_lock);
    reader_count--;
    if (reader_count == 0) {
        V(write_lock);          // 最后一个读者释放写锁
    }
    V(reader_count_lock);
}

```

写者逻辑：

```

void writer() {
    P(writer_count_lock);
    writer_count++;
    if (writer_count == 1) {
        P(read_try);            // 第一个写者"关门"
    }
}

```

```

V(writer_count_lock);

P(write_lock);          // 等待所有读者退出并开始写
write_data();           // 写入数据
V(write_lock);

P(writer_count_lock);
writer_count--;
if (writer_count == 0) {
    V(read_try);        // 最后一个写者"开门"
}
V(writer_count_lock);
}

```

关键点:

- `read_try` 是关键: 当有写者在等待时 (`writer_count > 0`), `read_try` 被第一个写者占用, 新读者无法通过 `P(read_try)` 进入
- 第一个写者锁住 `read_try`, 阻止新读者进入; 最后一个写者释放 `read_try`, 允许读者继续
- 写者必须等待所有已进入的读者退出后才能真正开始写 (通过 `P(write_lock)` 实现)
- `write_lock` 保证了读写互斥、写写互斥的核心逻辑

解决方案 (公平策略):

公平策略的目标是让读者和写者都不会"饥饿", 按照请求的先后顺序依次获得访问权。

信号量/变量作用说明:

读者逻辑:

```

void reader() {
    while (1) {

```


表 4: 公平策略中的信号量与变量作用

信号量/变量名称	作用说明
queue	全局队列锁（初值为 1），确保所有请求（读者和写者）按照先后顺序排队，防止插队现象
wrt	读写互斥锁（初值为 1），实现”读-写互斥”和”写-写互斥”的核心逻辑
mutex	保护 read_count 的互斥锁（初值为 1），确保读者计数的准确性
read_count	记录当前读者数量，用于管理”多读者并发”（当 read_count 为 0 时释放 wrt）

```

// 步骤1: 进入全局队列，保证FIFO顺序
P(queue);           // 排队（FIFO保证）
P(mutex);           // 保护read_count
read_count++;
if (read_count == 1) {
    P(wrt);          // 第一个读者申请wrt，阻止写者
}
V(mutex);           // 释放读者计数锁
V(queue);           // 释放队列锁，让后续请求排队

// 步骤2: 读取共享数据（多读者并发）
read_data();

// 步骤3: 最后一个读者释放wrt
P(mutex);           // 保护read_count
read_count--;
if (read_count == 0) {
    V(wrt);          // 最后一个读者释放wrt，允许写者
}

```

```

    }
    V(mutex);                // 释放读者计数锁
}
}

```

写者逻辑:

```

void writer() {
    while (1) {
        // 步骤1: 进入全局队列, 保证FIFO顺序
        P(queue);                // 排队 (FIFO保证)
        P(wrt);                  // 申请wrt, 独占资源 (写-写、读-写互斥)
        V(queue);                // 释放队列锁, 让后续请求排队

        // 步骤2: 修改共享数据 (写者独占)
        write_data();

        // 步骤3: 释放wrt, 允许下一个队列头部的请求
        V(wrt);
    }
}
}

```

关键点:

- `queue` (全局队列锁) 是公平策略的核心: 所有请求 (读者和写者) 都必须先通过 `P(queue)` 排队, 严格按照先来后到 (FIFO) 的顺序获得处理机会
- 读者在获得 `queue` 后立即释放 (`V(queue)`), 允许后续请求继续排队, 但只有当前读者批次完成后, 写者才能真正执行
- 写者在获得 `queue` 后同样立即释放, 但它会独占 `wrt`, 直到写操作完成, 此时下一个排队者才能继续
- `wrt` 信号量保证了核心互斥: 读-写互斥、写-写互斥, 只有第一个读者和所有写者需要申请它

- mutex 信号量仅用于保护 read_count 这一共享变量，防止多个读者同时修改计数导致数据不一致
- 公平性体现：无论是读者还是写者，都必须在 queue 队列中等待，避免了“读者优先”或“写者优先”导致的饥饿问题

5.3.3 哲学家就餐问题

问题简述：5 个哲学家围坐在圆形餐桌旁，每两个相邻哲学家之间放有 1 支筷子，共 5 支筷子。每个哲学家的行为只有“思考”和“就餐”两种。思考时不占用资源，就餐时必须同时拿起自己左右两边的两支筷子，且吃完后需将筷子放回原位。核心矛盾在于：哲学家要就餐必须获取 2 个相邻的资源（筷子），但资源总数有限且分配方式可能导致“循环等待”，进而引发死锁。

解决要点：防止死锁的发生，确保至少有一个哲学家能够同时获得两支筷子进行就餐。

解决方案：

解决方案（最多 4 人就餐）：

通过限制同时就餐的哲学家数量，确保至少有一支筷子空闲，避免死锁。

信号量定义：

- semaphore room = 4; // 房间容量，最多允许 4 人同时就餐
- semaphore chopstick[5] = {1, 1, 1, 1, 1}; // 5 支筷子，初值均为 1

哲学家进程逻辑：

```
void philosopher(int i) {
    while(1) {
        think();           // 思考
        P(room);           // 进入房间（限制人数）
        P(chopstick[i]);    // 拿起左边的筷子
        P(chopstick[(i+1) % 5]); // 拿起右边的筷子
```

```

        eat();                // 就餐
        V(chopstick[i]);      // 放下左边的筷子
        V(chopstick[(i+1) % 5]); // 放下右边的筷子
        V(room);              // 离开房间
    }
}

```

工作原理：

- 房间容量限制：最多允许 4 人同时进入房间（通过 room 信号量控制）
- 死锁避免逻辑：
 - 5 支筷子，最多 4 人持有筷子
 - 至少有 1 支筷子空闲
 - 至少有 1 人能同时拿到左右两支筷子
 - 该哲学家可以吃饭并最终释放筷子
- 资源释放：吃完后依次释放筷子和房间位置，其他哲学家可以继续

关键点：

- P(room) 必须在 P(chopstick) 之前，确保进入房间后才能拿筷子
- 通过限制并发数量（4 人）打破了”循环等待”条件，从根本上避免死锁

5.3.4 睡眠理发师问题

问题简述：理发店有 1 个理发师和若干个等待椅子。理发师如果没有顾客就睡觉；如果有顾客来就叫醒理发师并进行理发。如果所有等待椅子都满了，新的顾客就离开。需要确保理发师和顾客之间的同步与互斥。

解决方法：

解决方案：

理发店模型通过信号量机制协调”理发师”和”顾客”两类进程的同步与互斥，确保：

- 理发师在无顾客时睡眠，有顾客时被唤醒。
- 顾客到达时，若有空椅子则等待；若无空椅子则离开。
- 等待人数的修改需要互斥保护。

定义变量——记录核心状态

- `int waiting = 0;` 记录当前在等待区等待的顾客数量（需互斥保护）。
- `#define N 5`: 宏定义等待椅数量（示例中 $N = 5$ ，可根据实际场景调整）。

定义信号量——控制同步与互斥

表 5: 睡眠理发师问题的信号量设计

信号量名称	初值	作用
customers	0	顾客到达信号量: 用于唤醒睡眠的理发师, 初值为 0 (初始无顾客)。
barbers	0	理发师准备好信号量: 用于告知顾客理发师已空闲, 可开始理发, 初值为 0 (初始理发师可能睡眠, 未准备好)。
mutex	1	互斥信号量: 保护对”等待人数 (waiting)” 的修改, 确保同一时间只有 1 个进程 (顾客) 操作该变量。

理发师进程逻辑:

```
void barber() {
    while(1) {
        P(customers);           // 等待顾客到来 (若无顾客则睡眠)
        P(mutex);               // 保护waiting变量
        waiting--;              // 有顾客起身准备理发, 等待人数减1
```

```

        V(barbers);           // 通知顾客：理发师准备好了
        V(mutex);            // 释放互斥锁
        cut_hair();           // 理发（耗时操作）
    }
}

```

顾客进程逻辑：

```

void customer() {
    P(mutex);                // 保护waiting变量
    if (waiting < N) {        // 若有空椅子
        waiting++;            // 坐下等待，等待人数加1
        V(customers);         // 唤醒理发师（若睡眠）
        V(mutex);            // 释放互斥锁
        P(barbers);           // 等待理发师准备好
        get_haircut();         // 理发
    } else {                  // 若无空椅子
        V(mutex);            // 释放互斥锁
        leave();              // 直接离开
    }
}
}

```

关键点：

- `customers` 信号量实现了”顾客唤醒理发师”的同步机制
- `barbers` 信号量实现了”理发师通知顾客可以理发”的同步机制
- `mutex` 保护了共享变量 `waiting` 的互斥访问
- 顾客在发现无空椅子时直接离开，不会阻塞等待

5.4 进程通信 (IPC) 机制

管道 (Pipe)

管道本质上是内核在内存中维护的一个缓冲区，允许一个进程将数据写入管道，另一个进程从管道读取数据。

特性：自动同步（缓冲区满/空时阻塞）

比如最经典的命令行管道：`ls | grep "txt"`，其中 `ls` 的输出通过管道传递给 `grep` 进行过滤。输出后缀是 `.txt` 的文件列表。

信号 (Signal)

信号是一种异步通知机制，用于通知进程某个特定的事件已经发生。它类似于硬件中断，因此也被称为“软件中断”。

- 核心特点：
 - 开销极小：信号携带的信息量非常少，通常仅仅是一个整数（信号类型），不涉及大量数据传输。
 - 异步性：信号的产生是随机的（如用户按下键盘、硬件异常），进程不需要一直轮询，而是通过注册“信号处理函数”来被动响应。
 - 不可靠性：标准信号是不排队的，如果处理不过来可能会丢失（即“不排队”）。
- 典型场景：
 - 用户交互：当用户在终端按下 `Ctrl+C` 时，操作系统会向前台进程发送 `SIGINT` 信号，通常用于通知进程终止。
 - 进程控制：使用 `kill` 系统调用或命令发送信号（如 `SIGTERM` 或 `SIGKILL`）来管理或杀死进程。

消息队列 (Message Queue)

消息队列克服了管道只能传输无格式字节流的缺点，允许进程间传递结构化的数据。它本质上是保存在内核中的消息链表。

- 核心特点：
 - 结构化通信：消息具有边界和类型标识，发送的数据块是有格式的，不仅仅是连续的字节流。

- 选择性接收：这是与管道最大的区别。接收方可以根据消息类型（Type ID）选择接收特定的消息（例如优先处理紧急消息），而不必严格遵守先进先出（FIFO）。
- 异步性：发送方和接收方不需要建立实时的直接连接，消息可以暂存在队列中。

- 典型场景：

- 多级任务管理：例如打印机任务管理，可以将“普通任务”设为类型 1，“紧急任务”设为类型 2。驱动程序可以请求“只读取类型 2 的消息”，从而实现优先级插队。

共享内存 (Shared Memory)

共享内存允许两个或多个进程直接访问同一块物理内存区域。这是所有进程间通信（IPC）机制中速度最快的一种。

- 核心原理：

- 零拷贝 (Zero-Copy)：与其他机制（如管道、消息队列）不同，共享内存不需要将数据在“用户空间”和“内核空间”之间来回复制。进程 A 写入的数据，进程 B 立即就能看到。
- 地址映射：操作系统将同一块物理内存映射到不同进程的虚拟地址空间中。

- 关键挑战：同步问题

- 共享内存本身不提供任何同步互斥机制。
- 如果进程 A 正在写数据，进程 B 同时来读（或写），会导致数据错乱（竞态条件）。因此，共享内存必须配合信号量或互斥锁一起使用。

- 适用场景：

- 需要高性能且大数据量传输的场景（如实时视频处理、高频交易系统）。

6 死锁

6.1 基本概念：

6.1.1 定义与产生原因

什么是死锁：就是指这一组中的每个线程都在等待组内其他线程释放资源从而造成的无限等待

死锁产生的原因

死锁产生的根本原因可以归结为以下两点：

- 竞争不可抢占资源：
 - 系统中的资源数量有限（如打印机、文件锁），当多个进程竞争这些资源时，若资源不足以满足所有进程的需求，可能会导致死锁。
 - 具体表现为：当一个进程已占有了某些资源（如资源 A），同时又申请新的资源（如资源 B），而资源 B 正被其他进程占有且不可强制剥夺 (Non-preemptable) 时，该进程就会处于等待状态。如果对方也在等待资源 A，就会形成僵局。
- 进程推进顺序不当：
 - 进程在运行过程中，申请资源和释放资源的时机与顺序非常关键。
 - 如果进程按某种特定的、不当的顺序请求资源（例如形成循环等待链），就会导致死锁。反之，如果推进顺序合法（如按统一顺序申请），则不会发生死锁。

补充说明：死锁通常涉及至少两个进程，且与资源的特性（可重用资源如 CPU/内存，或可消耗资源如信号）紧密相关。

6.1.2 产生死锁的必要条件

- 互斥条件 (Mutual Exclusion)：在一段时间内某资源仅为一个进程所占有

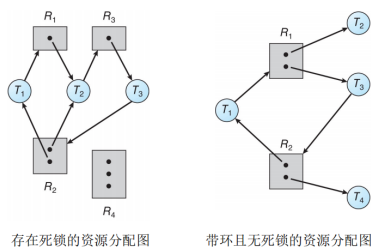


图 8: 资源分配图示例

- 持有并等待 (**Hold and Wait**): 又称部分分配条件。当进程因请求资源被阻塞时, 已分配资源保持不放
- 资源非抢占: 资源不能被强制从进程中剥夺, 只能由进程自己释放
- 循环等待 (**Circular Wait**): 存在一种进程资源的循环等待关系, 即 P_1 等待 P_2 占有的资源, P_2 等待 P_3 占有的资源, ..., P_n 等待 P_1 占有的资源

6.1.3 资源分配图

什么是资源分配图?

资源分配图 (Resource Allocation Graph, RAG) 是一种用于描述系统资源分配状态的有向图, 由节点集合 V 和边集合 E 组成。它是检测死锁的重要工具。

- 节点集合 V : 分为两类。
 - 进程节点集合 P : $\{P_1, P_2, \dots, P_n\}$, 在图中通常用圆形表示。
 - 资源节点集合 R : $\{R_1, R_2, \dots, R_m\}$, 在图中通常用方框表示。方框内的圆点数量代表该类资源的实例 (Instance) 数量。
- 边集合 E : 表示进程与资源之间的关系。
 - 请求边 (**Request Edge**): $P_i \rightarrow R_j$, 表示进程 P_i 正在申请资源 R_j 的一个实例 (箭头由进程指向资源)。

- 分配边 (**Assignment Edge**): $R_j \rightarrow P_i$, 表示资源 R_j 的一个实例已经分配给了进程 P_i (箭头由资源指向进程)。

死锁判定法则:

根据图中的环路情况, 可以判断系统是否处于死锁状态:

1. 无环: 如果资源分配图中没有环, 那么系统就没有进程死锁。
2. 有环: 如果分配图中存在环, 则需要根据资源实例数量进一步判断:
 - 每类资源仅有一个实例: 此时环路是死锁的充分必要条件, 存在死锁。
 - 每类资源有多个实例: 此时环路只是死锁的必要条件, 可能存在死锁。因为环中的某个进程可能只是在使用资源, 待其释放后环路即可解除 (如课件第 7 页图 2 所示)。

关键: 从资源分配图判断有没有死锁主要是观察当“仅仅持有资源”却不需要“被分配资源”的进程被释放后, 是否能打破环路!

6.2 处理策略

6.2.1 死锁预防: 破坏必要条件

死锁预防的核心思想是通过在系统设计时施加限制条件, 破坏导致死锁的四个必要条件之一, 从而防止死锁发生

- 破坏“互斥条件”
 - 原理: 试图让资源变为可共享的 (例如使用 SPOOLing 技术将独占的打印机改造为共享设备)。
 - 局限性: 大部分临界资源 (如内存写锁、磁带机) 在物理或逻辑上本质就是互斥的, 因此该策略适用范围非常有限。
- 破坏“占有并等待” (**Hold and Wait**)

- 实现方法：采用静态资源分配法。即进程在运行前必须一次性申请它所需要的全部资源。只有所有资源都满足时才运行，否则一直等待。
- 缺点：
 - * 资源利用率极低：有些资源可能最后才用，但进程一启动就一直占着。
 - * 饥饿现象：需要大量资源的进程可能一直凑不齐所有资源，导致长期无法运行。
- 破坏“不可抢占”(No Preemption)
 - 实现方法：当一个已经持有资源的进程申请新资源被阻塞时，它必须主动释放自己当前持有的所有资源。待以后重新获得旧资源和新资源时再继续执行。
 - 缺点：
 - * 实现复杂，需要保存和恢复进程状态。
 - * 仅适用于 CPU 寄存器或内存等易于保存状态的资源，不适用于打印机等设备（打印一半被打断会乱套）。
- 破坏“循环等待”(Circular Wait)
 - 实现方法：采用有序资源分配法。给系统中所有资源编号($R_1, R_2, \dots$)，规定进程必须按编号递增的顺序申请资源。如果已持有 R_i ，则只能申请 R_j (其中 $j > i$)。
 - 评价：这是目前最广泛采用的预防策略，相比前几种方法，其资源利用率较高，但会限制编程的灵活性。

6.2.2 死锁避免：银行家算法 (安全序列)

死锁的避免是在资源的动态分配过程中，用某种方法防止系统进入不安全状态，从而避免死锁的发生。

核心思想：银行家算法 (Banker's Algorithm) 通过在每次分配资源前，先检查分配后系统是否处于“安全状态”。只有确认安全后才真正分配，否则让进程等待。

关键概念：

- 安全状态：系统能找到一个进程执行序列 $\langle P_1, P_2, \dots, P_n \rangle$ ，使得每个进程都能依次获得所需资源、执行并释放资源，最终所有进程都能完成。
- 不安全状态：无法找到这样的序列，可能（但不一定）导致死锁。
- 安全序列：一个进程执行顺序，满足：对于序列中的每个进程 P_i ，它所需的剩余资源数量可以被“当前可用资源 + 之前所有进程释放的资源”满足。

数据结构定义：

假设系统有 n 个进程 $\{P_0, P_1, \dots, P_{n-1}\}$ ， m 类资源 $\{R_0, R_1, \dots, R_{m-1}\}$ ，定义以下数组：

1. **Available**：长度为 m 的向量，表示每类资源的剩余可用数量。

示例：Available[0]=3 表示第 0 类资源剩余 3 个。

2. **Max**： $n \times m$ 矩阵，表示每个进程对每类资源的最大需求量。

示例：Max[1][2]=5 表示进程 1 最多需要第 2 类资源 5 个。

3. **Allocation**： $n \times m$ 矩阵，表示每个进程已分配的每类资源数量。

示例：Allocation[0][1]=2 表示进程 0 已占用第 1 类资源 2 个。

4. **Need**： $n \times m$ 矩阵，表示每个进程还需要的每类资源数量。

计算公式：Need[i][j] = Max[i][j] - Allocation[i][j]

银行家算法执行步骤：

步骤 1：进程发起资源请求

进程 P_i 向系统请求资源，请求量用向量 Request _{i} 表示。

示例：Request₁ = [0, 2, 0] 表示进程 1 请求第 1 类资源 2 个。

步骤 2：合法性检查

- 检查是否超过最大需求：若 $\text{Request}_i[j] > \text{Need}[i][j]$ （对任意 j ），则请求非法，直接拒绝。

原因：进程申请的资源超过了它声明的最大需求，违反了系统约定。

- 检查系统资源是否充足：若 $\text{Request}_i[j] > \text{Available}[j]$ （对任意 j ），则系统剩余资源不足，进程需等待。

步骤 3：尝试”预分配”资源

系统临时修改数据结构，模拟分配后的状态（注意：此时并未真正分配）：

$$\text{Available}[j] \leftarrow \text{Available}[j] - \text{Request}_i[j]$$

$$\text{Allocation}[i][j] \leftarrow \text{Allocation}[i][j] + \text{Request}_i[j]$$

$$\text{Need}[i][j] \leftarrow \text{Need}[i][j] - \text{Request}_i[j]$$

步骤 4：执行安全性检查（寻找安全序列）

系统运行”安全性算法”，判断预分配后是否存在安全序列：

1. 初始化工作向量 $\text{Work} = \text{Available}$ ，标记向量 $\text{Finish}[i] = \text{false}$ （对所有 i ）。
2. 查找满足以下条件的进程 P_i ：
 - $\text{Finish}[i] = \text{false}$ （进程尚未完成）
 - $\text{Need}[i][j] \leq \text{Work}[j]$ （对所有 j ，即当前可用资源能满足该进程的剩余需求）
3. 若找到这样的进程 P_i ，则：
 - 假设进程 P_i 获得资源后能顺利执行并最终释放所有占用的资源。
 - 更新可用资源： $\text{Work}[j] \leftarrow \text{Work}[j] + \text{Allocation}[i][j]$
 - 标记进程已完成： $\text{Finish}[i] \leftarrow \text{true}$
 - 将 P_i 加入安全序列。

4. 重复步骤 2-3，直到：

- 情况 1：所有进程的 $\text{Finish}[i] = \text{true}$ ，说明找到了安全序列，系统处于安全状态。
- 情况 2：存在进程的 $\text{Finish}[i] = \text{false}$ 且无法继续找到满足条件的进程，说明系统进入不安全状态。

步骤 5：根据安全性检查结果做出决策

- 若系统安全：正式执行步骤 3 中的资源分配，进程 P_i 获得所需资源。
- 若系统不安全：回滚步骤 3 的预分配操作，恢复 Available、Allocation 和 Need 的原值，拒绝此次请求，进程 P_i 需等待。

示例：

假设系统有 5 个进程 $\{P_0, P_1, P_2, P_3, P_4\}$ ，3 类资源 $\{A, B, C\}$ 。初始状态如下：

进程	Max			Allocation			Need		
	A	B	C	A	B	C	A	B	C
P_0	7	5	3	0	1	0	7	4	3
P_1	3	2	2	2	0	0	1	2	2
P_2	9	0	2	3	0	2	6	0	0
P_3	2	2	2	2	1	1	0	1	1
P_4	4	3	3	0	0	2	4	3	1

当前可用资源：Available = [3, 3, 2]

初始安全性检查：按照安全性算法，可以找到安全序列 $\langle P_1, P_3, P_4, P_2, P_0 \rangle$ ，系统处于安全状态。

现在 P_1 请求资源：Request₁ = [1, 0, 2]

检查过程：

1. 检查 Request₁ ≤ Need₁：[1, 0, 2] ≤ [1, 2, 2]
2. 检查 Request₁ ≤ Available：[1, 0, 2] ≤ [3, 3, 2]

3. 预分配后：

- $Available = [3 - 1, 3 - 0, 2 - 2] = [2, 3, 0]$
- $Allocation_1 = [2 + 1, 0 + 0, 0 + 2] = [3, 0, 2]$
- $Need_1 = [1 - 1, 2 - 0, 2 - 2] = [0, 2, 0]$

4. 执行安全性检查，若能找到安全序列则分配；否则拒绝并回滚。

总结：银行家算法通过“预分配 + 安全性检查”的方式，在每次资源分配前进行预判，确保系统始终处于安全状态，从而有效避免死锁的发生。

6.2.3 死锁检测：化简资源分配图

具体参考上面资源分配图部分内容、

6.2.4 死锁解除

当检测到死锁发生时，系统需要采取措施解除死锁，通常采用以下两种方法：

1. 进程终止 (Process Termination)

- 终止所有死锁进程：最简单的方法，但代价极大，因为这些进程已经完成的计算工作将全部丢失。
- 逐个终止进程：每次只终止一个死锁进程，然后再次调用死锁检测算法，直到死锁消失。
- 选择策略：为了最小化代价，在选择“牺牲者”时通常考虑以下因素：进程优先级、已运行时间、已占用资源数量以及还需要的资源数量。

2. 资源抢占 (Resource Preemption)

- 选择牺牲者 (Select Victim)：选择一个代价最小的进程，强行剥夺其持有的资源分配给其他进程。

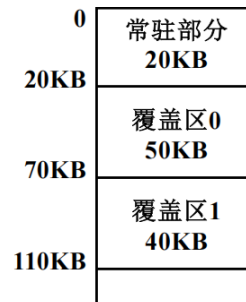
- 回滚 (**Rollback**): 被剥夺资源的进程无法继续正常执行, 必须将其回滚到某个安全状态 (最简单的是完全重启该进程)。
- 防止饥饿 (**Starvation**): 必须确保同一个进程不会总是被选为牺牲者 (例如, 在计算代价时考虑该进程被回滚的次数), 否则该进程可能永远无法完成。

7 内存管理

7.1 基础概念

- 逻辑地址: 这是程序视角下的地址, 也称为相对地址, 程序通常认为自己是从地址 0 开始存放的
- 物理地址: 这是硬件实现视角下的地址, 也就是数据在内存条上实际存储的绝对位置, 如果程序的基址 (起始位置) 是 1000, 逻辑地址 500 对应的物理地址实际上是 1500
- 重定位技术: 将程序的逻辑地址转换为物理地址的技术
 - 静态重定位 (**Static Relocation**): 编译器或加载器直接将程序中的指令地址修改为绝对物理地址, 由软件完成。缺点是程序一旦加载到内存就不能移动, 不支持多道程序运行。
 - 动态重定位 (**Dynamic Relocation**): 在程序运行时进行地址转换, 依赖于重定位寄存器 (基址寄存器) 的硬件支持。计算公式: 物理地址 = 逻辑地址 + 基址寄存器的值。优点是程序可以加载到内存的任意位置, 运行时可以在内存中移动。
- 内存保护 (Memory Protection): 使用界限寄存器 (Limit Register) 防止越界访问。检查条件为: 逻辑地址 < 界限寄存器值。若不满足则触发越界异常; 若满足则加上基址寄存器的值形成物理地址。
- 覆盖与交换技术
 1. 覆盖技术 (Overlay)

- (a) 核心场景：针对单个程序。当程序代码量大于物理内存时的解决方案
- (b) 基本原理：程序员手动切分程序为若干功能模块。常驻区存放核心代码并始终在内存中；覆盖区存放互斥模块（不同时执行），共用同一内存区域。



- (c) 例子：程序总大小 190KB，内存 110KB。常驻区放 A 模块 (20KB)；覆盖区 0 放 B(50KB) 和 C(30KB) 互斥；覆盖区 1 放 D、E、F 中最大的 E(40KB)；总需求 $20+50+40 = 110\text{KB}$ 。
- (d) 优点：解决大程序在小内存上运行
- (e) 缺点：程序员必须手动规划模块调用关系，难度大

2. 交换技术 (Swapping)

- (a) 核心场景：针对多个程序（多道程序环境）。当内存里挤满了进程，新的进程进不来，或者被阻塞的进程占着茅坑不拉屎时，怎么办
- (b) 基本原理：操作系统充当”调度员”，把暂时不用的进程整个搬到硬盘（外存）里，腾出地方给急需运行的进程。换出 (Swap Out) 是指进程从内存移到硬盘对换区；换入 (Swap In) 是指进程从硬盘搬回内存运行。
- (c) 特点：透明性强，支持多道程序，整体交换
- (d) 缺点：硬盘 I/O 开销大，性能较低

7.2 内存分配方案

7.2.1 连续分配

1. 固定分区分配 (Fixed Partitioning):

- 原理：将内存划分为若干固定大小的分区，每个分区可以容纳一个进程
- 优点：实现简单，易于管理
- 缺点：内存利用率低，可能产生大量碎片; 数量固定; 大小死板

2. 动态分区分配 (Dynamic Partitioning):

按程序需求分配不预先划分分区大小，存储结构有空闲分区表和空闲分区链表两种示意图如下：



分区号	大小	起始地址
1	8KB	24KB
2	12KB	128KB
3	8KB	248KB
4	...	...
5	...	...

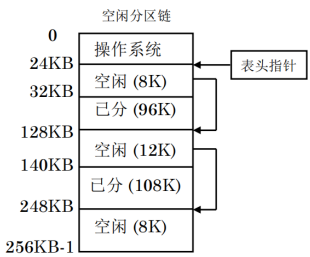


图 9: 内存布局图

图 10: 空闲分区表

图 11: 空闲分区链

但是动态分区方案依然存在一个问题那就是外部碎片问题 (外部碎片是指内存中存在许多小的空闲分区，这些分区的总量虽然足以满足一个新作业的申请要求，但由于它们在物理位置上是不连续的 (被已分配的作业隔开了)，导致无法将该作业装入内存) 有如下两种解决方案:

- (a) 紧凑化 (Compaction): 移动所有进程-> 开销极大
- (b) 分配算法优化:

- 首次适应算法 (First Fit): 从头开始查找第一个满足要求的空闲分区 (特点: 优先利用内存低地址端, 高地址端有大空闲区。但低地址端有许多小空闲分区时会增加查找开销)
- 循环首次适应算法 (Next Fit): 从上次分配结束的位置开始查找 (特点: 使存储空间的利用更加均衡, 但会使系统缺乏大的空闲分区)
- 最佳适应算法 (Best Fit): 查找所有满足要求的空闲分区, 选择最小的一个 (特点: 减少内存浪费, 但会产生大量小的不可用碎片)
- 最差适应算法 (Worst Fit): 查找所有满足要求的空闲分区, 选择最大的一个 (特点: 剩下的空闲区比较大, 但当大作业到来时, 其存储空间的申请往往得不到满足)

对于算法的衡量好坏的标准: 对于某一个作业序列来说, 若某种分配算法能将该作业序列中所有作业安置完毕, 则称该分配算法对这一作业序列合适, 否则称为不合适

3. 伙伴系统 (Buddy System): 采用伙伴算法对空闲内存进行管理该方法通过不断对分大的空闲存储块来获得小的空闲存储块。当内存块释放时, 应尽可能合并空闲块。基本原理:

- 内存块大小均为 2^k 的形式
- 分配时, 找到最小的满足需求的 2^k 块
- 若找到的块大于需求, 则不断对半拆分, 直到接近需求大小
- 释放时, 检查相邻伙伴块 (大小相同) 是否空闲, 若是则合并

优缺点:

- 优点: 回收效率高: 合并算法简单快速; 灵活性: 比固定分区更灵活, 且避免了动态分区中复杂的“紧凑”操作
- 缺点: 内部碎片: 由于分配块必须是 2 的幂次, 若申请 65KB 需分配 128KB, 会产生明显的内存浪费; 拆分/合并开销: 频繁的分配与回收会导致多次拆分与合并操作

例子：假设初始内存为 1MB，作业申请序列为 A(200K)、B(120K)。分配 A(200K) 时分配 256K 块，剩余 768K；分配 B(120K) 时分配 128K 块，剩余 640K；释放 A 时释放 256K 块，检查伙伴后无法合并。

7.2.2 离散分配

1. 分页管理 (Paging):

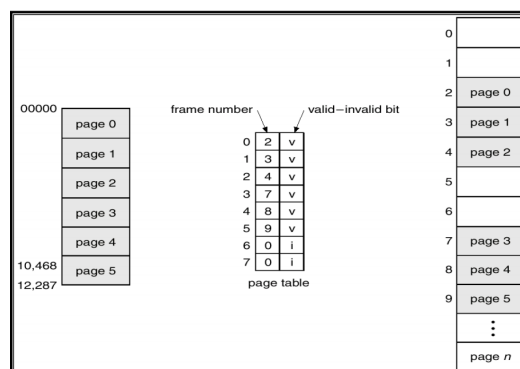


图 12: 分页管理示意图

页表 (如图 12): 逻辑页面-> 物理块页面-> 页框 (就是将进程划分为若干个固定大小的页，每页大小通常为 4KB，然后将这些页映射到物理内存中的页框)

计算地址:

$$\text{物理地址} = \text{页框号} \times \text{页大小} + \text{页内偏移}$$

快表: 并行查找缓存最近使用的页表项，提高地址转换速度

有效时间计算: 给出一个例子吧: 假设内存一次存取时间是 m ，联想寄存器的查找时间是 n ，命中率为 p 那么，有效存取时间 EAT 为:

$$EAT = (n + m) \times p + (n + 2m) \times (1 - p) = n + m + (1 - p) \times m$$

解释: 命中时需要查找快表和访问内存，未命中时需要查找快表、访问页表和访问内存两次

不同的分页管理方式:

表 6: 页表与快表比较

特性	页表 (Page Table)	快表 (TLB)
位置	主内存 (RAM)	CPU 内部 (MMU)
介质	DRAM (普通存储器)	SRAM (联想存储器)
容量	大, 存放全部页表项	小, 通常只有 64-1024 项
访问速度	慢	极快
查找方式	索引查找	并行比较查找

- 多级页表: 将页表划分为多级结构, 减少内存占用

例子: 假设逻辑地址为 32 位, 页面大小为 4KB (2 的 12 次方), 则页内偏移占 12 位, 剩余 20 位用于页号。

页面大小 $\rightarrow 4\text{KB} = 2^{12}$ bytes $\rightarrow$ 页内偏移 12 位 $\rightarrow$ 剩余 20 位用于页号

将页号划分为两级: 高 10 位作为一级页表索引, 低 10 位作为二级页表索引 (因为每个页表项长度是 4 字节, 所以每个页表可以包含 2^{10} 个页表项)

- 哈希页表: 使用哈希函数映射逻辑页号到页表项, 适用于大地址空间 (哈希函数 $\rightarrow$ 逻辑页号 (哈希链表中顺序查找) $\rightarrow$ 物理块号)

例子: 假设逻辑页号为 **12345**, 哈希表大小为 **1024**, 哈希函数为 $h(x) = x \bmod 1024$, 则 $h(12345) = 12345 \bmod 1024 = 57$, 查找哈希表索引 **57** 处的链表以找到对应的页表项. **57** $\rightarrow$ (**12345**, **5**) $\rightarrow$ (**6789**, **12**) $\rightarrow$...

- 反向页表: 直接按序查找物理块对应的逻辑页, 节省内存

2. 分段管理 (Segmentation): 段是逻辑划分的结构, 按程序需求分配不预先划分分区大小, 存储结构有段表, 每个段有段号、段基址、段界限等信息

3. 段页式管理 (Segmented Paging): 就是先用分段管理将程序划分为若干段, 然后对每个段再进行分页管理

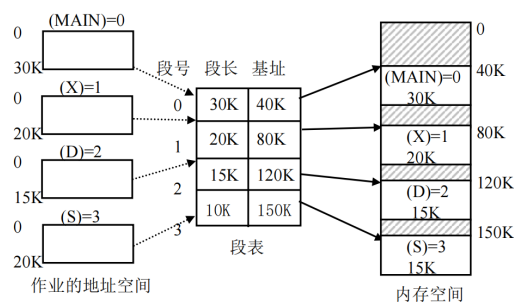


图 13: 分段管理示意图

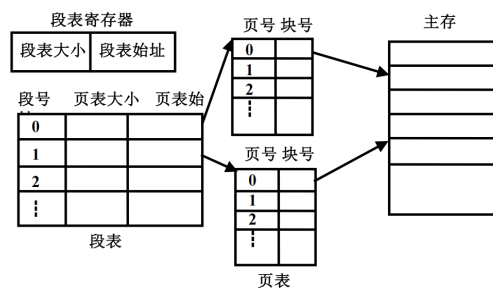


图 14: 段页式管理示意图

7.3 虚拟内存技术

虚拟内存技术: 将用户逻辑内存与物理内存分开, 使得程序可以使用比实际物理内存更大的地址空间

局部性原理: 程序在执行过程中, 在较短的一段时间内, 所执行的指令地址和操作数地址往往集中在一定的存储区域内

- 时间局部性: 最近访问过的数据和指令在不久的将来可能会被再次访问 (循环结构 → 带来时间局部性。)
- 空间局部性: 与最近访问过的数据和指令地址相近的数据和指令在不久的将来可能会被访问 (顺序执行 → 带来空间局部性。)

虚拟存储器的实现方法:

- 请求分页 (Demand Paging): 只有在页面被访问时才加载到内存
- (a) 扩充后的页表结构:

- 有效位 (Valid Bit): 指示该页是否在内存中
- 修改位 (Dirty Bit): 指示该页是否被修改过
- 访问位 (Accessed Bit): 指示该页是否被访问过
- 页号和物理块号
- 外存位置指针: 指向该页在外存中的位置

(b) 页错误:

- i. 发生条件: 访问的页不在内存中 (有效位为 0)
- ii. 处理步骤:
 - A. 检查页表以确定引用是否合法
 - B. 引用非法则终止, 有效但不在内存则调入
 - C. 找到一个空闲帧
 - D. 把页换入页框
 - E. 修改页表, 使其有
 - F. 重启指令

(c) 缺页中断: 就是处理页错误的过程

- (d) 有效访问时间 (EAT) 计算: 内存的读写周期为 t , 缺页中断服务时间为 t_1 (包含读入缺页、页表更新、快表更新时间) 快表的命中率为 α , 缺页中断率为 f , 快表访问时间为 ε , 则有效存取时间可表示为

$$EAT = \alpha \times (t + \varepsilon) + (1 - \alpha) \times [(1 - f) \times 2(t + \varepsilon) + f \times (t_1 + 2(t + \varepsilon))]$$

为什么是 $2(t + \varepsilon)$ 呢? 因为未命中快表时, 需要先访问页表 (一次内存访问), 然后再访问实际数据页 (第二次内存访问), 对于快表则是访问 + 修改

(e) 页面置换算法:

- FIFO: 先入先出, 缺点: 可能会淘汰掉经常使用的页面
- OPT: 最佳置换, 缺点: 需要预知未来访问情况, 实际难以实现
- LRU: 最近最少使用, 优点: 较好地利用了局部性原理, 缺点: 实现复杂, 需要维护访问时间信息

- LRU 近似算法: 使用访问位, 定期清除访问位, 选择未被访问的页面进行置换
- CLOCK: 时钟算法, 使用一个指针循环扫描页面, 选择访问位为 0 的页面进行置换
- 改进时钟算法: 结合修改位, 优先选择访问位和修改位都为 0 的页面进行置换
- LFU: 最不经常使用, 选择访问次数最少的页面进行置换

(f) 页面分配算法

- 平均分配: 将物理内存均匀分配给所有进程
- 按比例分配: 根据进程大小按比例分配内存
- 按优先级分配: 分为两部分, 先按比例分配, 再根据进程优先级分配内存

(g) 抖动 (Thrashing):

如果运行进程的大部分时间都用于页面的换入/换出, 而几乎不能完成任何有效的工作, 则称此进程处于抖动状态。抖动又称为颠簸、颤动。

产生原因:

- 进程分配的物理块太少
- 置换算法选择不当
- 全局置换使抖动传播

(h) 工作集模型 (Working Set Model):

- 定义: 在某一时间段内, 进程实际使用的页面集合
- 工作集窗口 (Δ): 定义为最近 Δ 时间内访问的页面集合
- 工作集大小 (WSS): 工作集窗口内的页面数量
- 目标: 保持进程的工作集在内存中, 减少缺页率

举个例子: 观察窗口 (Δ): 管理员盯着你看了 10 分钟 (这就是工作集窗口); 工作集 (WSS): 他发现这 10 分钟里, 你反反复复翻阅的其实只有《数学》、《物理》、《化学》这 3 本书
决策: 虽然你有 100 本参考书, 但在当前阶段, 只要给你书桌上腾出 3 个位置, 你就能顺畅工作了

- 请求分段 (Demand Segmentation):

只有在段被访问时才加载到内存。请求分段的主要优势包括：有利于动态增长、允许按段进行编译、有利于共享和保护。

7.4 优化技术

写时复制 (**COW**): 只要没人要修改数据，大家就共享同一份；一旦有人要修改，再给他单独复制一份

这个过程利用了“只读”标记和“欺骗”手段：

1. 初始状态：父子进程共享同一页，页表项标记为“只读”
2. 修改请求：当某个进程尝试写入该页时，CPU 发现该页面是“只读”的，于是报错（触发异常/中断）
3. 处理异常：操作系统捕获这个中断，发现是因为“写时拷贝”机制引起的，于是它说：“好吧，既然你要改，那我现在给你复制一份。”
4. 结果：现在，进程 1 拥有了自己的新页面，进程 2 还在用旧页面。只有被修改的那一页发生了复制，其他没改的页依然共享。

8 文件系统

8.1 文件系统基础

- 扇区：磁盘上最小的存储单位，通常为 512 字节磁盘的盘片被划分成许多同心圆（磁道），每个磁道又被切分成许多小弧段，这些小弧段就是“扇区”
- 元数据：即描述文件属性的信息，而不是文件内容本身，元数据通常包含以下内容：
 1. 标识 (文件名/类型/Inode)
 2. 位置 (文件在磁盘上的存储地址)

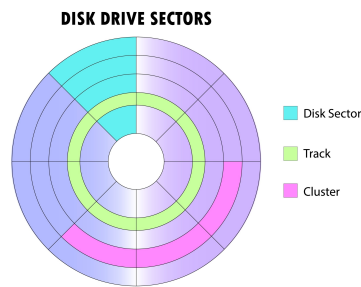


图 15: 扇区示意图

3. 属性 (权限/所有者/创建时间/修改时间/大小)
 4. 时间
 5. Inode: 一种数据结构, 用于存储文件的元数据和磁盘块地址 (固定 256 字节)
 6. Dirent: 目录项, 包含文件名和对应的 Inode 号
- 视图转换: 视图转换操作主要依靠 **Inode** (提供静态索引地图) 和 **bmap** (提供动态查找算法) 进行实现。其目的是将用户眼中的“连续字节流 (逻辑视图)”转换为磁盘眼中的“离散块 (物理视图)”。
- 工作链条: 文件名 $\xrightarrow{\text{查目录 (dirent)}}$ Inode 号 $\xrightarrow{\text{查 Inode 表}}$ Inode 结构体 $\xrightarrow{\text{查 Addr 数组}}$ 物理块号 $\xrightarrow{\text{读磁盘}}$ 文件数据
- 加入 **bmap** 后 (细化逻辑块到物理块的映射): 在多级索引结构中, 简单的“查 Addr 数组”被扩展为由 **bmap** 函数执行的复杂计算过程:
1. 逻辑坐标计算: 系统根据用户提供的字节偏移量算出逻辑块号 (LBN)。逻辑块号 (LBN) = $\lfloor \text{字节偏移量} / \text{块大小} \rfloor$
 2. **bmap** 映射逻辑: 内核调用 `bmap(inode, LBN)`, 根据 LBN 的大小在 Inode 的混合索引表中顺藤摸瓜:
 - 直接寻址: 若 $LBN < 12$, 则 物理块号 (PBN) = `Inode.Addr[LBN]`。
 - 一次间接: 若 $12 \leq LBN < 1036$, 则 **bmap** 读取 `Inode.Addr[12]` 指向的索引块并查找。

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	0	0	1	1	0	1	1	1	0	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	0	0	0	0	0	0	1
2	1	1	1	1	1	1	0	1	1	1	1	0	0	0	0	0
3																
4	...															
⋮																

图 16: 位图法示意图

- 多次间接：若 LBN 更大，bmap 需逐级访问二级或三级间接块指针（Inode.Addr[13] 或 [14]），最终锁定 PBN。

3. 最终物理寻址：(Inode + LBN) $\xrightarrow{\text{bmap 算法}}$ 物理块号 (PBN) $\xrightarrow{\text{驱动读取}}$ 物理扇区数据

8.2 文件组织方式

8.2.1 空闲空间管理

1. 位图法 (Bitmap Method):

用一个二进制位（0 或 1）代表一个物理块的状态。0 表示空闲，1 表示已分配。

- 优点：容易找到连续的空闲块。
- 缺点：当磁盘很大时，位图本身会占用大量内存。

2. 空闲链表法 (Linked List Method):

把所有空闲块连成一串链表，每个空闲块包含一个指向下一个空闲块的指针。

- 优点：分配简单。
- 缺点：如果要申请大量块，需要沿着链表一个一个找，效率极低。

3. 成组链接法 (Grouped Link Method):

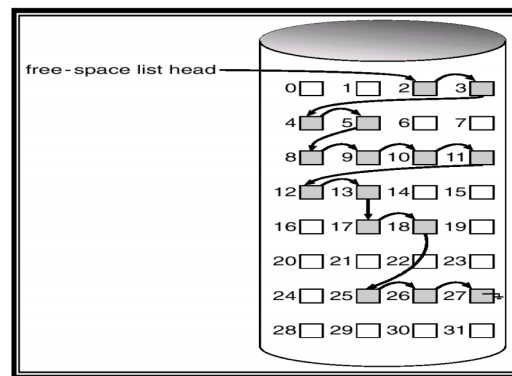


图 17: 空闲链表法示意图

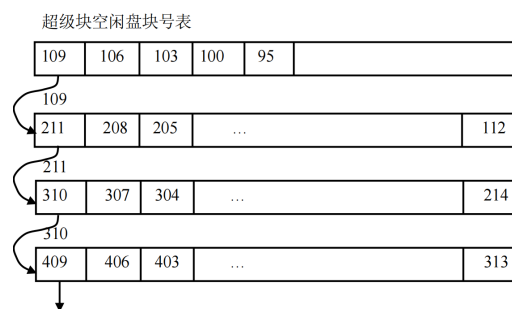


图 18: 成组链接法示意图

将所有空闲块按组划分, 每一组的第一个块会记录下一组所有空闲块的块号建立了一种“层层递进”的结构。

概念理解:

- 超级块: 它是整个文件系统的总指挥官, 它固定在磁盘分区的最开头 (比如 0 号或 1 号块), 永远不动。它记录了整个分区的基本信息
- 超级块空闲盘块号表: 超级块中有一个专门的区域, 用来存放一组空闲盘块的块号, 这个区域就叫做“超级块空闲盘块号表”。它就像一个小型的目录, 告诉系统哪些盘块是空闲的, 可以用来存储新文件或数据。

8.2.2 接口设计

文件描述符作为凭证, Open 方法建立连接, Close 方法断开连接。

1. 文件描述符 (File Descriptor, fd)

- 定义: 一个非负整数 (如 3, 4, 5...)。
- 作用: 它是用户进程与内核文件结构之间的抽象句柄。用户不需要接触底层的 Inode 或物理地址, 只需持有这个“号码牌”即可操作文件。
- 本质: 它是当前进程“打开文件表”数组的下标索引。

你的进程里有一个指针数组 `files[ ]`, 当你 `open` 一个文件时, 内核找一个空闲的位置 (比如下标 3), 让 `files[3]` 指向内核里的一个打开文件对象

然后函数返回 3 给你的程序, 下次你调用 `read(3, ...)`, 操作系统直接去查 `files[3]`, 顺藤摸瓜找到文件。

3 就是 fd。

2. Open 方法

- 功能: 打开指定路径的文件, 建立读写会话。

- 原型: `int Open(char *path, int mode);`
- 内部流程:
 - (a) 寻址 (**Path Lookup**): 根据路径查找目录, 找到文件的静态 Inode (身份证)。
 - (b) 安检 (**Check**): 检查用户权限 (读/写) 是否合法。
 - (c) 建档 (**State Creation**): 在内核中创建一个动态的 file 结构体, 初始化读写指针 (**offset**) 为 0。
 - (d) 发牌 (**Allocation**): 在进程的文件描述符表中找到一个空闲位置 (如下标 3), 指向上述结构体, 并将该下标作为 **fd** 返回。

3. Close 方法

- 功能: 关闭文件, 释放资源。
- 原型: `int Close(int fd);`
- 内部流程:
 - (a) 回收号码牌: 清空进程文件描述符表中 **fd** 对应的项, 使其可以被再次分配。
 - (b) 销毁档案: 减少内核中对应 file 结构体的引用计数。若计数归零, 则释放该结构体占用的内存。
 - (c) 释放关联: 断开与底层 Inode 的动态连接。

8.2.3 连续分配

连续分配方法要求每个文件在磁盘上占有一组连续的块

- 特点: 顺序访问容易且速度快, 目录中文件存储位置信息简单; 但容易产生碎片, 需要定期对磁盘空间进行整理
- 存在的问题: 为新文件找空间比较困难; 文件很难增长

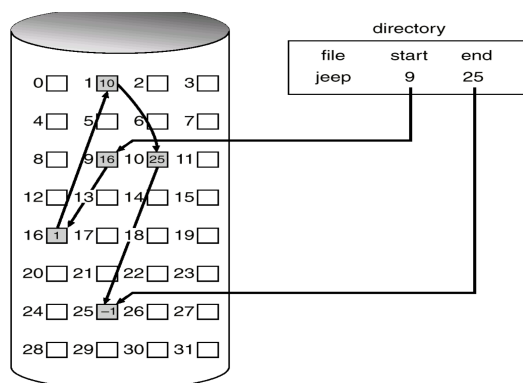


图 19: 链接分配示意图

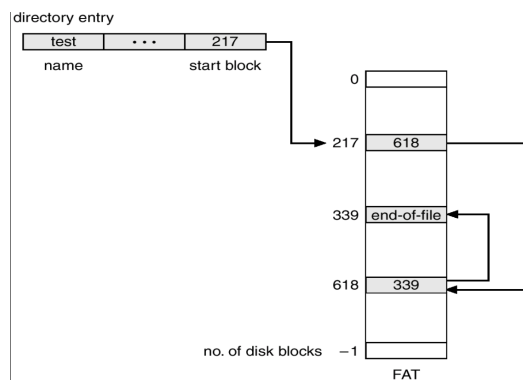


图 20: FAT 文件分配表示意图

8.2.4 链接分配

以扇区为单位的链接分配/以区段（或簇）为单位的链接分配

- 优点：简单—只需起始位置；文件创建与增长容易
- 缺点：不能随机访问；块与块之间的链接指针需要占用空间；存在可靠性问题，如指针损坏

文件分配表 (FAT) 就是一种典型的以扇区为单位的链接分配方法

该表整个文件系统仅设一张，其结构如图所示。表的序号是物理块号，从 0 开始直至 $N - 1$ (N 为盘块总数)

$$\text{FAT 表大小} = \text{磁盘块总数} \times \text{每个表项的大小}$$

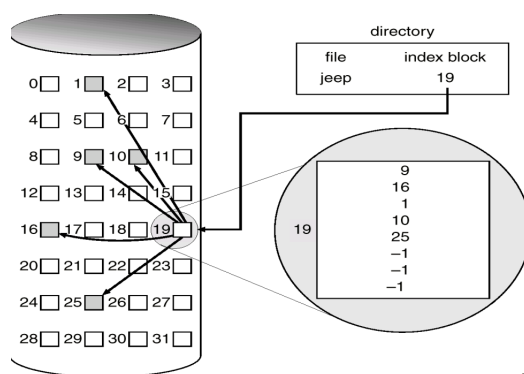


图 21: 索引分配示意图

对于 磁盘块总数 就是磁盘空间/ 磁盘块大小

对于 每个表项的大小 盘块越多，编号就越长，记录这个编号所需的空
间就越大, 因为 1 字节 (Byte) = 8 位 (bit) 所以计算方法是：

$$\text{每个表项的大小 (字节)} = \lceil \log_2(\text{磁盘块总数}) / 8 \rceil$$

8.2.5 索引分配

索引分配概念：为每个文件建立一个索引块，索引块中存放该文件所有数据块的地址，实现随机访问

- 优点：索引分配方法支持直接访问，不会产生外部碎片
- 缺点：但索引块要占用一定的存储空间，存取文件需要两次访问外存

example: 假设有一个文件 ABC.txt，内容分散在磁盘的第 8, 2, 5 号块中，目录项 (FCB)：记录文件名 ABC.txt，并记录索引块的地址，索引块 (第 100 号)：里面写着三个数字：8, 2, 5。用户想读文件的第 3 部分，系统找到第 100 号块（索引块），直接读取数组的第 3 项 → 发现是 5，磁头直接飞到 5 号块读取数据。

地址：

假设盘块大小为 4KB，一个盘块号占 4 字节

1. 直接地址：索引节点 (Inode) 中直接存放数据块的物理盘块号。

直接地址根本就没有给你“分配一个专门的块”来存钥匙，它只是在 Inode 里借了几个位置而已

寻址范围：通常假设 Inode 含 10 个直接地址项，则范围为 $10 \times 4KB = 40KB$ 。

2. 一次间接地址：索引节点指向一个索引块，该块中存放指向数据块的地址（即存放 $4KB/4B = 1024$ 个地址）。

寻址范围： $1024 \times 4KB = 4MB$ 。

3. 二次间接地址：索引节点指向一个一级索引块，一级块指向 1024 个二级索引块，二级块再指向数据块。

寻址范围： $1024 \times 1024 \times 4KB = 1M \times 4KB = 4GB$ 。

8.3 文件目录与路径

8.3.1 文件链接

概念：允许一个文件拥有多个名字，或者让一个文件指向另一个文件。这让用户和程序可以从不同的路径访问同一个文件，极大提高了文件组织的灵活性

1. 硬链接 (**Hard Link**)：硬链接是指多个不同的目录项（文件名）直接指向同一个索引节点（Inode）

采取的措施：引用计数机制（少一个链接，计数减一，计数为 0 时删除文件）

限制：不能跨文件系统、不能链接目录

2. 软链接 / 符号链接 (**Symbolic Link**)：软链接是一个独立的文件，包含指向另一个文件路径的文本字符串。不同文件有不同的 Inode

优点：可以跨文件系统、可以链接目录

3. 比较：为什么硬链接不能链接目录？

假设你有一个目录 /A/B，如果你在 B 下面创建了一个指向 A 的硬链接 hard_link_to_A

结果：路径变成了 /A/B/hard_link_to_A/B/hard_link_to_A/... 这就形成了一个环路 (Cycle) 。

8.4 文件操作接口

挂载：将外部文件系统的根目录连接到已有目录树的某个节点。
实现步骤：

- 1. 读取超级块：验证文件系统类型 (超级快: 文件系统大小、块大小、空闲 Inode 数、空闲块数、魔数（标识文件系统类型）)
- 2. 连接根目录：挂载点 dentry → 新文件系统根 Inode
- 3. 标记挂载点：d_mounted 标志

8.5 典型文件系统

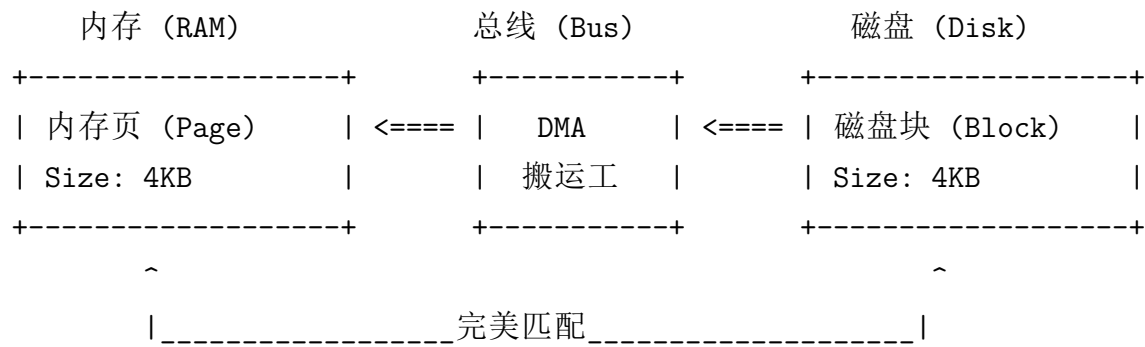
表 7: UNIX 与 FAT32 文件系统特性对比

特性	UNIX 文件系统	FAT32 文件系统
元数据位置	Inode (与文件名分离)	目录项 (包含所有信息)
物理分配方式	多级索引分配 (树状)	显式链接分配 (链式)
单文件上限	TB 级 (几乎无限制)	4GB (受限于 32 位 size 字段)
长文件名支持	原生支持	需 VFAT 扩展支持
可靠性	高 (引用计数、日志等)	中 (主要依靠 FAT 表备份)
实现复杂度	高	低
典型应用场景	Linux/macOS、服务器	U 盘、SD 卡、嵌入式设备

8.6 文件系统性能

1. 缓存技术 (**Caching**): 文件系统之所以需要“缓存技术”，核心原因只有一个：磁盘太慢了，而内存和 CPU 太快了。缓存技术就是把最近访问的数据保留在内存中，避免频繁访问慢速磁盘，从而提升整体性能。

- 缓冲区缓存 (Buffer Cache): 缓存磁盘块，减少物理读写次数 (单位：磁盘块 (4KB))
- 页面缓存 (Page Cache): 缓存文件数据页，提高文件读写效率 (单位：内存页 (4KB))



2. 预读与写回 (**Read-Ahead and Write-Back**): 原理：检测顺序访问，提前读取后续块
- 初始 4-8 块，连续命中：则窗口扩大，随机访问：则窗口缩小
3. 延迟写 (**Delayed Write**): 将写操作先缓存在内存中，待条件满足时再批量写入磁盘，减少磁盘写入次数，提高效率

8.7 虚拟文件系统 (VFS)

VFS 是一层软件抽象层，它位于应用程序和具体的文件系统实现之间，提供统一的文件系统接口

设计原因：Linux 支持的文件系统非常多（ext4, xfs, fat32, nfs...），它们的底层实现各不相同。FAT32 使用文件分配表，没有 Inode，Ext4 使用 Inode 和多级索引。如果程序员写代码时，读 FAT32 文件要用 `read_fat()`，读 Ext4 要用 `read_ext4()`，那简直是噩梦。

VFS 定义了一套统一的标准接口（POSIX 标准，如 `open`, `read`, `write`）。应用程序只管调用这些标准接口，VFS 负责把它们“翻译”成底层文件系统能听懂指令

作用：统一接口，屏蔽底层差异

四大核心数据结构：

- 超级块 (**super_block**)：表示一个挂载的文件系统，包含文件系统类型、大小、状态等信息
- 索引节点 (**inode**)：表示文件的元数据，如权限、所有者、时间戳、数据块位置等
- 目录项 (**dentry**)：表示目录中的一个文件或子目录，包含文件名和对应的 inode 号
- 文件对象 (**file**)：表示一个打开的文件，包含读写指针、访问模式等信息

有很多人会弄混 Inode 和 fd, 这里给出一个例子进行说明：

比如描述一个看电影的事情，这个电影存放在磁盘中，我打开是由 Inode 定位到这个“静态资源”，但是我现在有两个人观看，打开两次分配了两个 fd (1,2)，作为文件描述符，比如一个人 a 看了 10 分钟，一个人 b 看了 1 分钟，a 如果暂停后再看会调用 fd=1 回到 10min 处而不是 1 分钟。

9 设备管理

9.1 基本概念

I/O 设备及其分类：I/O 设备（Input/Output Devices，输入/输出设备）是计算机系统中除了 CPU（大脑）和内存（短期记忆）以外的所有外部设备的总称

1. 按使用特性:

- 人机交互设备: 键盘、鼠标、显示器
- 存储设备: 硬盘、U 盘、光驱
- 网络通信设备: 网卡、调制解调器

2. 按传输速率:

- 低速: 键盘 (10B/s)、鼠标 (100B/s)
- 中速: 打印机 (100KB/s)
- 高速: 磁盘 (100MB/s)、网卡 (1GB/s)

3. 按信息交换单位:

- 块设备: 硬盘、U 盘 (可随机访问)
- 字符设备: 键盘、串口 (流式访问)
- 网络设备: 网卡 (数据包)

I/O 端口: 要了解 I/O 端口, 首先要明白什么是设备控制器。设备控制器就像是设备的“翻译官”, 它负责把计算机发出的数字信号转换成设备能理解的形式, 反之亦然。

一个 I/O 设备就是通常由机械部件 (物理设备) 和电子部件 (控制器) 组成的。I/O 端口就是设备控制器与 CPU 或内存之间进行数据交换的接口

端口类型	作用
数据端口 (Data)	存放输入/输出数据 (缓冲)
状态端口 (Status)	指示设备忙/闲、错误 (握手)
控制端口 (Ctrl)	接收启动、复位等命令

I/O 控制方式: 解决“超快的 CPU”和“超慢的 I/O 设备”之间的速度矛盾, 尽可能把 CPU 从繁重的 I/O 搬运工作中“解放”出来

1. 程序控制方式 (Programmed I/O): CPU 不断地去查看设备控制器的状态寄存器。CPU 问: “你好了吗?”, 设备: “没”。CPU 再问: “好了吗?”, 设备: “没”……直到设备说“好了”, CPU 才去读写数据

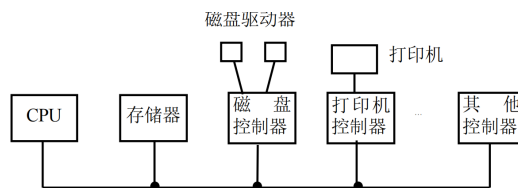


图 22: 总线示意图

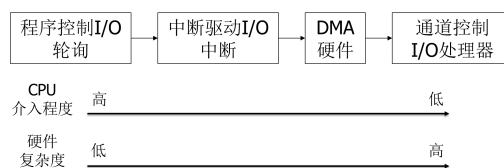


图 23: I/O 控制方式示意图

- 优点：实现简单，适用于高速设备
 - 缺点：CPU 忙于轮询，效率低下
2. 中断驱动方式 (Interrupt-Driven I/O): CPU 发出指令让设备开始工作，然后 CPU 转头去干别的事。当设备做完了，会发一个中断信号（就像敲门或电话）告诉 CPU。CPU 停下手头工作，去处理数据
- 优点：提高 CPU 利用率，适用于中速设备
 - 缺点：每次中断都需要 CPU 介入，开销较大
 - 适用：键盘、鼠标等低中速设备
3. DMA 方式 (Direct Memory Access): DMA 控制器是一个专门的搬运工，CPU 只需告诉它“去把数据从设备搬到内存的哪个位置”，然后就可以继续干别的事了。DMA 控制器自己完成搬运任务，发送中断通知 CPU
- 优点：大幅减少 CPU 介入，适合大批量数据传输
 - 适用：磁盘网卡等高速设备
4. 通道控制方式 (Channel I/O): 通道控制器是一个更高级的搬运工，它不仅能搬运数据，还能执行一系列复杂的 I/O 操作指令。CPU 给通

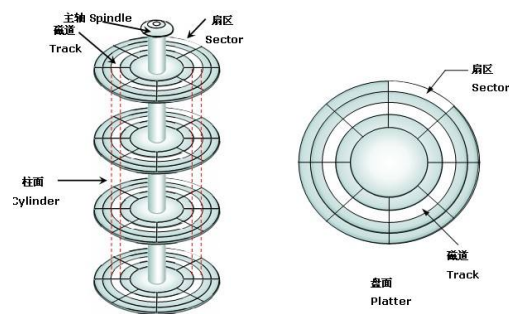


图 24: 机械硬盘结构示意图

道发送一段“通道程序”（任务清单）。通道自主执行多个 I/O 操作，统筹管理多台设备，甚至可以进行简单的逻辑处理

- 优点：极大减轻 CPU 负担，适用于大型机
- 适用：大型机系统

9.2 硬盘

9.2.1 机械硬盘

机械硬盘 (HDD) 是一种传统的存储设备，利用磁性材料在旋转的盘片上存储数据。

总访问时间 = 寻道时间 + 旋转延迟 + 传输时间

性能瓶颈：机械运动，寻道和旋转导致访问延迟高

9.2.2 固态硬盘

SSD 内部没有任何机械运动部件，它更像是一个巨大的、断电不丢数据的内存条 组成：

- NAND 闪存芯片：存储数据的主要介质
- 控制器：SSD 的“大脑”，负责管理数据读写和内部算法
- DRAM 缓存：提高读写速度

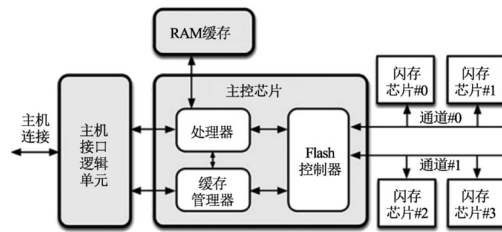


图 25: 固态硬盘结构示意图

特性:

- 读写单位是“页” (Page): 你可以一次只读或写 4KB 的数据
- 擦除单位是“块” (Block): 你不能只删除某一个页, 要删就得把整个块 (几百个页) 全部擦除
- 写前必须擦除 (Erase before Write)

SSD 中还有一个非常核心的概念——**FTL (Flash Translation Layer, 闪存转换层)**

FTL 是固态硬盘 (SSD) 控制器内部的一个核心软件层, 它的主要作用是解决操作系统 (基于逻辑块地址 LBA) 与 NAND Flash 物理特性 (基于物理页地址 PPA, 且写前必须擦除) 之间的矛盾

三大功能和一大问题:

1. 地址映射 (Address Mapping): 将逻辑块地址 (LBA) 映射到物理页地址 (PPA)
2. 垃圾回收 (Garbage Collection): 回收无效数据页, 整理存储空间
3. 磨损均衡 (Wear Leveling): 均匀分配写入次数, 延长 NAND Flash 寿命
4. 问题: 写放大效应 (Write Amplification): 实际写入的数据量大于请求写入的数据量, 影响性能和寿命

机械硬盘 (HDD) 与固态硬盘 (SSD) 对比:

内存 (RAM) 与磁盘 (Disk/Storage) 对比:

对比维度	机械硬盘 (HDD)	固态硬盘 (SSD)	核心差异点
存储介质	磁性盘片 (Magnetic Platter)	NAND Flash 芯片	物理原理不同 (磁 vs 电)
内部结构	机械结构: 马达、盘片、磁头臂	纯电子结构: 控制器、Flash 芯片	HDD 有活动部件, SSD 无
随机访问速度	慢 (~10 ms)	极快 (~0.1 ms)	SSD 快约 100 倍 (核心优势)
顺序读写速度	一般 (100–200 MB/s)	极快 (500–3500 MB/s)	SSD 拷贝大文件更快
抗震性	差 (怕摔, 有机械部件)	优秀 (抗震, 无机械部件)	SSD 更适合笔记本/移动设备
功耗与噪音	高 (5–10W), 有马达噪音	低 (2–4W), 完全静音	SSD 更省电、安静
使用寿命	物理磨损 (寿命较长)	P/E 擦写次数有限	SSD 写多了会坏 (但家用足够)
价格成本	低 (适合存大量冷数据)	高 (适合装系统和软件)	HDD 单位容量更便宜

9.2.3 磁盘调度算法

假设当前磁头位置为: 53

请求队列: 98, 183, 37, 122, 14, 124, 65, 67

1. 先来先服务 (FCFS):

53 → 98 → 183 → 37 → 122 → 14 → 124 → 65 → 67

总寻道距离 = $|53-98| + |98-183| + |183-37| + |37-122| + |122-14| + |14-124| + |124-65| + |65-67| = 640$

2. 最短寻道时间优先 (SSTF): 找和当前磁头位置距离最近的请求

53 → 65 → 67 → 37 → 14 → 98 → 122 → 124 → 183

3. 扫描算法 (SCAN): 磁头会沿着一个方向 (比如从小到大) 一直移动, 处理沿途的所有请求, 直到碰到磁盘的物理边界 (0 号磁道或最大磁

对比维度	内存 (Memory / RAM)	磁盘 (Disk / Storage)	通俗比喻
技术类型	DRAM (动态随机存取存储器)	HDD (磁性) 或 SSD (闪存)	工位桌面 vs 仓库书架
系统角色	CPU 的“工作台”，数据必须先加载到这里 CPU 才能处理	数据的“仓库”，负责长期保存文件和数据	桌上正在看的书 vs 书架上放着的书
速度差异	最快 (纳秒级)，比 SSD 还要快几十上百倍	较慢 (毫秒/微秒级)，即使是 SSD 也比内存慢得多	伸手拿文件 vs 走路去仓库取
断电保存	易失性 (Volatile)，断电/关机数据全丢	非易失性 (Non-Volatile)，断电后数据依然保留	下班清空桌面 vs 存入档案柜
典型容量	小 (8GB-64GB)	大 (512GB-10TB+)	桌面空间有限 vs 仓库很大
解决卡顿	解决多任务卡顿、程序无响应 (增大桌面，能同时干更多事)	解决开机慢、加载慢 (提升从仓库取货的速度)	针对不同瓶颈：RAM 提升并行，磁盘升级提升存取

道号)，才掉头往回走

53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 14 → 37

4. **LOOK** 算法: 磁头沿一个方向移动，处理沿途的请求，但不会跑到磁盘边界，而是在没有请求的地方就掉头 53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37
5. 循环扫描算法 (**C-SCAN**): 磁头沿一个方向移动，处理沿途的请求，跑到磁盘边界后直接跳回另一端，继续处理请求 53 → 65 → 67 → 98 → 122 → 124 → 183 → 199 → 0 → 14 → 37
6. 循环 **LOOK** 算法 (**C-LOOK**): 磁头沿一个方向移动，处理沿途的请求，跑到最后一个请求后直接跳回最前面的请求，继续处理 53 → 65 → 67 → 98 → 122 → 124 → 183 → 14 → 37

7. **N-Step-SCAN** 算法: 将请求队列分成若干个长度为 N 的小队列, 磁头每次只处理一个小队列, 处理完后再处理下一个小队列

示例路径(假设 $N = 4$): 我们将请求队列 {98, 183, 37, 122, 14, 124, 65, 67} 分为两组:

- 子队列 1: {98, 183, 37, 122}
- 子队列 2: {14, 124, 65, 67}

第一步(处理子队列 1): 从 53 开始, 按 SCAN (向大方向): $53 \rightarrow 98 \rightarrow 122 \rightarrow 183 \rightarrow (\text{掉头}) \rightarrow 37$

第二步(处理子队列 2): 从 37 开始, 按 SCAN (接力方向): $37 \rightarrow 14 \rightarrow (\text{掉头}) \rightarrow 65 \rightarrow 67 \rightarrow 124$

完整路径: $53 \rightarrow 98 \rightarrow 122 \rightarrow 183 \rightarrow 37 \rightarrow 14 \rightarrow 65 \rightarrow 67 \rightarrow 124$

8. **FSCAN** 算法: 核心逻辑: FSCAN 是 N-Step-SCAN 的简化版, 它只使用 ** 两个子队列 **。

- ** 当前队列 **: 包含当前所有需要处理的请求, 由磁盘调度按 **SCAN** 算法进行处理。
- ** 缓冲队列 **: 在扫描期间新出现的所有请求, 都会被放入这个缓冲队列, 暂不处理。
- 当“当前队列”处理完后, 系统才会切换去处理“缓冲队列”。这能有效防止磁臂粘着, 保证所有请求最终都能被处理。

示例路径: 由于本题给出的请求队列是 ** 静态 ** 的(即假设所有请求在开始时都已存在, 且期间没有新请求插入), FSCAN 的行为将与标准的 SCAN 算法一致, 因为它会把这 8 个请求全部视为“当前队列”并锁定。

$53 \rightarrow 65 \rightarrow 67 \rightarrow 98 \rightarrow 122 \rightarrow 124 \rightarrow 183 \rightarrow 199 \rightarrow 14 \rightarrow 37$

注: 如果处理过程中有新请求(如 55)到达, 它不会被立即插队处理, 而是会留到下一轮扫描。

9.2.4 分区与格式化

分区：

1. MBR (Master Boot Record):

- 最大支持 2TB 磁盘
- 最多支持 4 个主分区
- 分区表存储在磁盘的第一个扇区（512 字节）

2. GPT (GUID Partition Table):

- 支持超过 2TB 的大容量磁盘
- 支持最多 128 个分区
- 分区表存储在磁盘的多个位置，提供冗余备份

格式化：

分完区后，房间（分区）虽然有了，但里面还是空的，没有货架，也没有索引，货物扔进去就找不到了。格式化就是为了解决数据的管理问题。它分为两类

1. 低级格式化 (Low-Level Formatting):

这是物理层面的操作。它在磁盘的磁性表面上划分磁道（Track）和扇区（Sector），并打上电子标记

2. 高级格式化 (High-Level Formatting):

建立文件系统

9.3 时钟

时钟四大作用：

1. 维护系统时间（当前时间）
2. 驱动进程调度（时间片）

3. 实现定时功能 (sleep、timeout)

4. 统计 CPU 使用时间

硬件定时器和软定时器：

没有硬件定时器（挂钟）提供时间基准，软定时器（日程表）就无法运作

硬件定时器就像墙上的挂钟。它只负责机械地“滴答、滴答”走字，客观且永不停歇。软定时器就像你的日程表。你看着墙上的挂钟（硬件定时器），发现到了 9:00，于是去开会；发现到了 12:00，于是去吃饭

表 8: 硬件定时器与软定时器对比

特性	硬件定时器 (Hardware Timer)	软定时器 (Software Timer)
本质	物理设备 (电路/芯片)	内核功能 (数据结构/代码)
来源	由晶振、分频器和计数器芯片组成	基于硬件中断，由操作系统内核模拟
功能	产生周期性的物理中断信号 (Tick)	在特定时间点触发回调函数 (Function)
精度	极高，由晶振频率决定	受限于系统中断频率 (HZ) 和系统负载
角色	系统的”脉搏”	应用程序的”闹钟”

时钟中断：硬件定时器每隔固定时间产生中断

时钟中断就是硬件定时器每隔几毫秒给 CPU 打的一个“骚扰电话”。CPU 接起电话（响应中断），迅速地更新系统时间、检查闹钟，并看看当前进程是不是该下台了，从而维持整个操作系统的秩序

硬件链路：晶振（产生震荡）→ 分频器（降频）→ 定时器 → 中断控制器 → CPU

动作：硬件定时器会按照预设的时间间隔（比如每 1ms），无条件地给 CPU 发送一个中断信号，强迫 CPU 停下手头的活来处理这个信号

9.4 I/O 系统层次架构

1. 硬件设备层：包括各种物理 I/O 设备，如磁盘、键盘、显示器等。
2. 中断处理程序层：负责响应硬件设备发出的中断请求，通知操作系统有 I/O 事件发生。

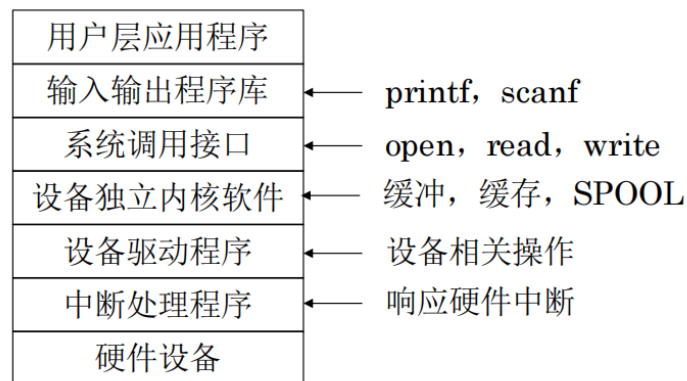


图 26: I/O 系统层次架构示意图

3. 设备驱动程序层：为每种具体的 I/O 设备提供操作接口，封装设备的硬件细节。
4. 设备独立内核软件层：提供统一的 I/O 接口，屏蔽不同设备驱动的差异。
5. 系统调用接口层：为用户进程提供标准的 I/O 操作接口，如 `read`、`write` 等。
6. 输入输出程序库层：为应用程序提供更高级的 I/O 功能封装，如文件操作库。

9.5 中断处理程序

9.5.1 上下文保护与恢复

保存内容：

- 程序计数器 (PC / IP)：记录了“被打断时执行到了哪行代码”（存盘点）
- 程序状态字 (PSW / EFLAGS)：记录了当前的运算状态（比如是否进位、是否为零、当前权限等级）
- 通用寄存器 (EAX, EBX, ECX...)：记录了程序正在计算的中间变量

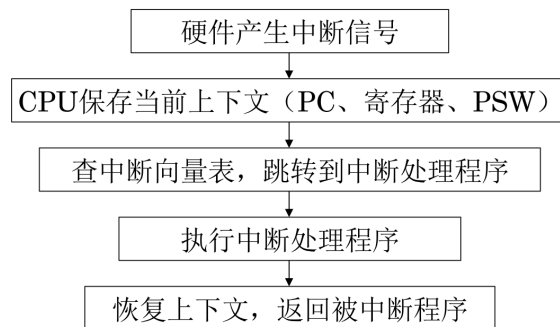


图 27: 中断处理程序示意图

- 栈指针 (SP): 记录了当前函数调用栈的位置

保存位置: 栈空间 (内核栈)

保存方式:

1. 硬件自动保存: CPU 在响应中断时, 自动将 PC 和 PSW 压入内核栈
2. 软件手动保存: 中断处理程序开始时, 手动将通用寄存器和 SP 压入内核栈

中断嵌套: 处理中断 A 时, 又发生中断 B, 如果 B 优先级高, 可以打断 A

嵌套造成的问题:

- 栈溢出风险: 每次嵌套都要保存上下文
- 实时性影响: 低优先级长时间得不到服务
- 复杂性增加: 临界区保护更复杂

9.5.2 上半部与下半部机制

为什么要分离:

如果不分两半: CPU 一口气把所有事情做完 (比如处理网络包、写硬盘)。这会耗时很久, 导致 CPU 长时间 “失聪”, 丢失其他重要中断, 系统卡顿

- 中断处理应尽可能快
- 某些工作可以延迟
- 避免在中断上下文做复杂操作

于是我们分为上下两部分

表 9: 上半部与下半部机制对比

特性	上半部 (Top Half)	下半部 (Bottom Half)
别名	硬中断 (Hardirq)	软中断 (Softirq) / 延迟处理
执行时机	中断发生后立刻执行	稍后执行 (系统空闲或中断退出前)
中断状态	关闭中断 (CPU 听不到别的)	开启中断 (CPU 可以响应新中断)
主要任务	1. 签收硬件信号 (Ack) 2. 拷贝少量数据 3. 登记下半部 (告诉 OS 有活要干)	1. 复杂的数据处理 2. 协议栈解析 3. 磁盘 I/O 操作
耗时要求	极快 (微秒级)	可以较长
能否睡眠	绝对禁止 (会导致死锁)	Workqueue 方式可以睡眠

9.6 设备独立内核软件技术

缓冲区 (Buffer) 是一块临时存储区域, 用于在数据传输过程中暂时存放数据, 以弥补生产者和消费者之间的速度差异

举个例子: CPU 是“法拉利”, I/O 设备 (如打印机、硬盘) 是“老牛车”, 如果没有缓冲, 法拉利跑一米就得等老牛车走一米。有了缓冲, CPU 可以把一堆数据扔进缓冲区 (仓库), 然后去干别的, 让 I/O 设备慢慢搬。如果没有缓冲, 每传输 1 个字节就要打断一次 CPU, 有了缓冲, 凑满 4KB (一个缓冲区大小) 才打断一次 CPU, 效率大增

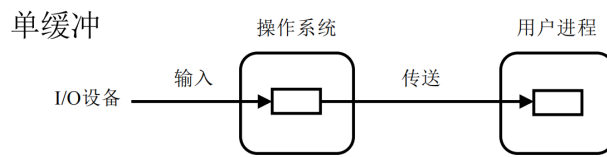


图 28: 单缓冲示意图

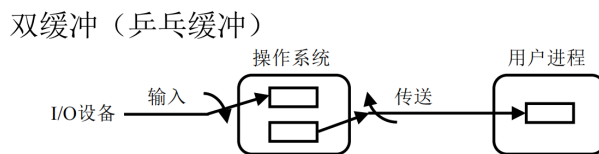


图 29: 双缓冲示意图

9.6.1 缓冲技术

单缓冲:

- 输入阶段: 设备把数据写进缓冲区 (此时 CPU 不能动这个缓冲区, 必须等待)。
- 传送阶段: 缓冲区满了, 操作系统把数据从缓冲区搬到用户区。
- 算阶段: CPU 开始处理用户区的数据。

计算公式: 假设: $M =$, $C = CPU$ 处理一块数据的时间 $= M + C$
设备和 CPU 不能同时工作

双缓冲:

- 设备往 1 号缓冲区倒数据。
- CPU 同时从 2 号缓冲区取数据。
- 当 1 号满了、2 号空了, 他俩交换目标。设备写 2 号, CPU 读 1 号。

计算公式: $\max(C, M)$ 设备和 CPU 可以同时工作

环形缓冲:

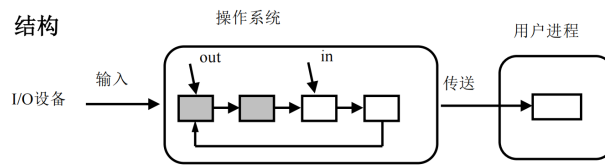


图 30: 环形缓冲示意图

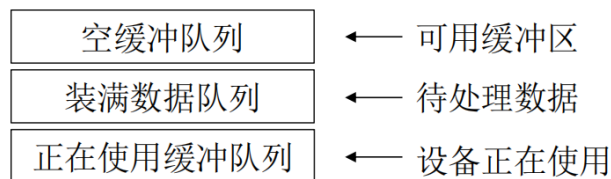


图 31: 缓冲池示意图

输入指针 (In): 指向下一个空缓冲区 (给设备装货用) 输出指针 (Out): 指向下一个满缓冲区 (给 CPU 卸货用)

空: $out == in$ 满: $(in + 1)$

- 充分利用缓冲区
- 适合流式数据 (音频、串口)

缓冲池:

缓冲池由多个缓冲区组成, 每个缓冲区都有一个缓冲首部 (记录身份信息, 如设备号、状态) 和缓冲体 (真正存数据的地方) 灵活高效

- 空闲缓冲队列: 这里面全是空的缓冲区, 谁想用都可以来拿
- 输入队列: 这里面全是装满了数据的缓冲区。这些数据是从设备读进来的, 正在排队等着 CPU (用户进程) 来取
- 输出队列: 这里面也全是装满了数据的缓冲区。但这些数据是 CPU 算出来的, 正在排队等着输出到设备上去

工作流程:

1. 设备从空队列获取缓冲区

- 2. 填充数据后放入装满队列
- 3. CPU 从装满队列取出处理
- 4. 处理完放回空队列

缓冲缓存的对比：

表 10: 缓冲与缓存对比		
维度	缓冲 (Buffer)	缓存 (Cache)
英文	Buffer	Cache
核心目的	平滑速度差异 (协调快慢)	加速数据访问 (减少延迟)
数据使用	通常一次	可能多次
方向	单向 (设备到内存或内存到设备)	双向 (CPU 与内存之间)
出现时机	数据在传输/移动过程中	数据在反复使用过程中
典型策略	FIFO (先进先出), 满了就送走	LRU (最近最少使用), 热点保留, 冷门淘汰
生活比喻	卸货区的传送带	图书馆的热门书架

9.6.2 异步 I/O (AIO)

异步 I/O 允许程序发起 I/O 请求后立即返回，无需等待 I/O 完成，从而提高系统的并发处理能力。

工作流程 (以 `aio_read()` 函数为例)：

- 1. 发起请求：程序调用异步 I/O 函数（如 `aio_read()`），立即返回，不阻塞。
- 2. 继续执行：程序继续执行后续代码，处理其他任务。
- 3. I/O 完成：操作系统完成 I/O 操作后，通过信号或回调函数通知程序。
- 4. 处理结果：程序在回调函数中处理 I/O 结果。

表 11: 同步 I/O 与异步 I/O 对比

维度	同步阻塞 I/O (Blocking I/O)	异步 I/O (Asynchronous I/O)
调用后行为	线程被挂起 (Blocked), 必须等待 I/O 完成	函数立即返回, 线程继续执行其他任务
CPU 状态	空闲等待, 切换到其他进程	持续工作, 执行后续代码
通知机制	I/O 完成后线程被唤醒	通过信号 (Signal) 或回调函数 (Callback) 通知
资源消耗	高并发时需要大量线程 (每个连接一个线程)	少量线程即可处理大量并发请求
编程复杂度	简单直观 (顺序执行)	较复杂 (需要回调处理、状态管理)
典型函数	<code>read()</code> , <code>write()</code>	<code>aio_read()</code> , <code>aio_write()</code>
生活比喻	站在柜台前傻等面做好	拿取餐器离开, 面好了震动通知
适用场景	简单、低并发应用	高并发服务器 (Web、数据库)

优缺点:

- 优点:
 - 提高 CPU 利用率
 - 适合 I/O 密集型应用
- 缺点: —> 非重点
 - 编程模型复杂, 需要处理回调和状态
 - 调试困难, 错误不易追踪
 - 不是所有操作系统都完全支持

9.6.3 SPOOLing 技术 (假脱机)

SPOOLing 是操作系统中为了解决“独占设备”（如打印机）无法被多个进程共享，以及“CPU 与外设速度差异过大”这两个核心矛盾而诞生的技术

SPOOLing 工作原理（以打印为例）：

场景对比：

- 没有 SPOOLing 时：

1. Word 申请占用打印机，打印机被独占。
2. Word 发送第 1 页 → 打印机打印（耗时约 5 秒）。
3. 在此期间，Word 界面卡死，CPU 必须等待，无法切换到其他任务。
4. 发送第 2 页... 直到 100 页全部打印完成，Word 才恢复响应，打印机才释放。
5. 其他进程（如 Excel）想打印？必须排队等待。

- 有了 SPOOLing 后：

1. Word 点击打印。
2. SPOOLing 程序立即将这 100 页数据快速保存到硬盘的”输出井”（缓冲区）中。
3. SPOOLing 告诉 Word：”打印任务已接收！”（实际未真正打印，只是存储）。
4. Word 瞬间恢复响应，用户可以继续编辑文档或执行其他操作。
5. 操作系统在后台由 SPOOLing 输出进程慢慢地将硬盘中的数据发送给打印机。
6. 其他进程的打印任务也可以同时提交到输出井，排队依次处理。

SPOOLing 系统组成：

1. 输入井 / 输出井：磁盘上的缓冲区，存放等待输入/输出的数据。

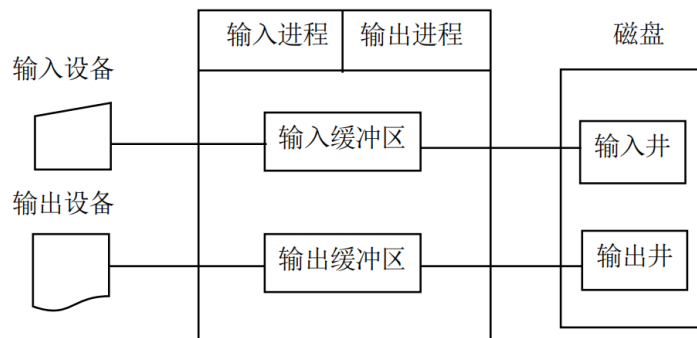


图 32: SPOOLing 系统结构示意图

2. 输入缓冲区 / 输出缓冲区：内存中的高速缓冲，加速数据传输。

3. 输入进程 / 输出进程：负责在井与缓冲区之间搬运数据。

4. 请求表：记录所有待处理的 I/O 请求及其状态。

优缺点：——> 非重点

优点：

- 提高设备利用率：将独占设备改造为虚拟的共享设备。
- 提高 CPU 效率：进程无需等待慢速设备，立即返回继续执行。
- 实现虚拟设备：一台物理打印机可以被多个进程“同时”使用（实际是排队）。

缺点：

- 需要占用额外的磁盘空间作为缓冲。
- 增加了系统复杂度，需要额外的进程调度和管理。

9.6.4 设备分配与回收

设备分类：

1. 独占设备：一次只能被一个进程使用（如打印机）

2. 共享设备：可同时被多个进程使用（如磁盘）
3. 虚拟设备：通过 SPOOLing 技术将独占设备虚拟化为共享设备

分配策略：

- 先请求先服务 (FCFS)：按照请求顺序分配设备
- 优先级调度：根据进程优先级分配设备
- 死锁避免：在分配设备时考虑避免死锁的发生

设备独立性：

设备独立性是指应用程序使用逻辑设备名来请求设备，由操作系统将逻辑设备名映射到物理设备名，从而使应用程序独立于具体的物理设备。

示例：应用程序请求”打印机”，操作系统将其映射到 /dev/lp0（具体的物理打印机）。

保护机制：

操作系统通过权限检查来保护设备资源，防止未授权访问。

在 Linux 系统中，设备文件具有与普通文件相同的权限机制：

- 读权限 (r)：允许从设备读取数据
- 写权限 (w)：允许向设备写入数据
- 执行权限 (x)：对设备文件通常无意义

示例：crw-rw---- 1 root lp /dev/lp0

- c：字符设备
- rw-：所有者（root）可读写
- rw-：组用户（lp 组）可读写
- ---：其他用户无权限